UNIVERSITÄT
LEIPZIG

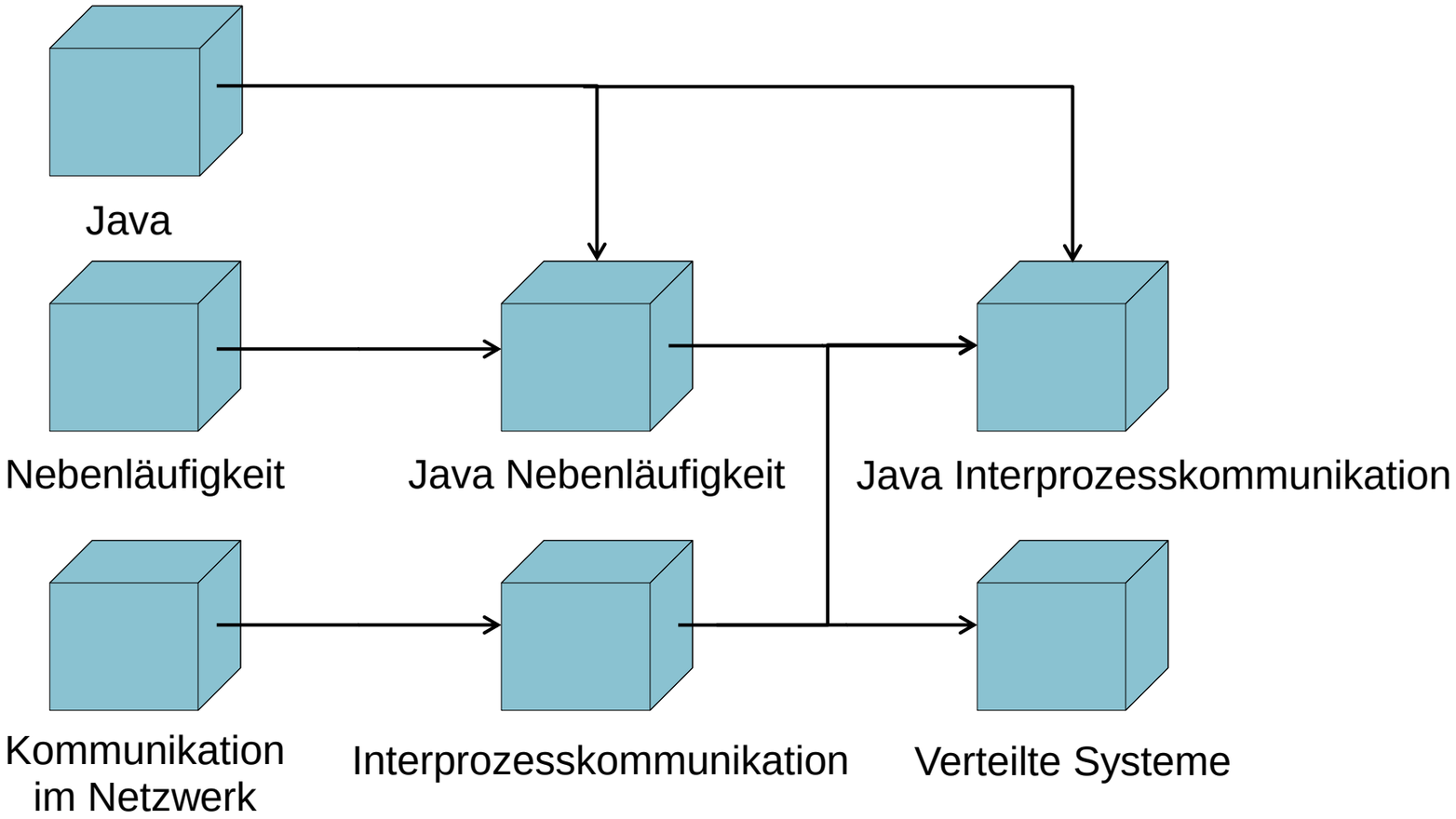
Middleware

Entwicklung verteilter Anwendungen

Leipzig, 16.06.2021

Dr. Pascal Kovacs

AGENDA

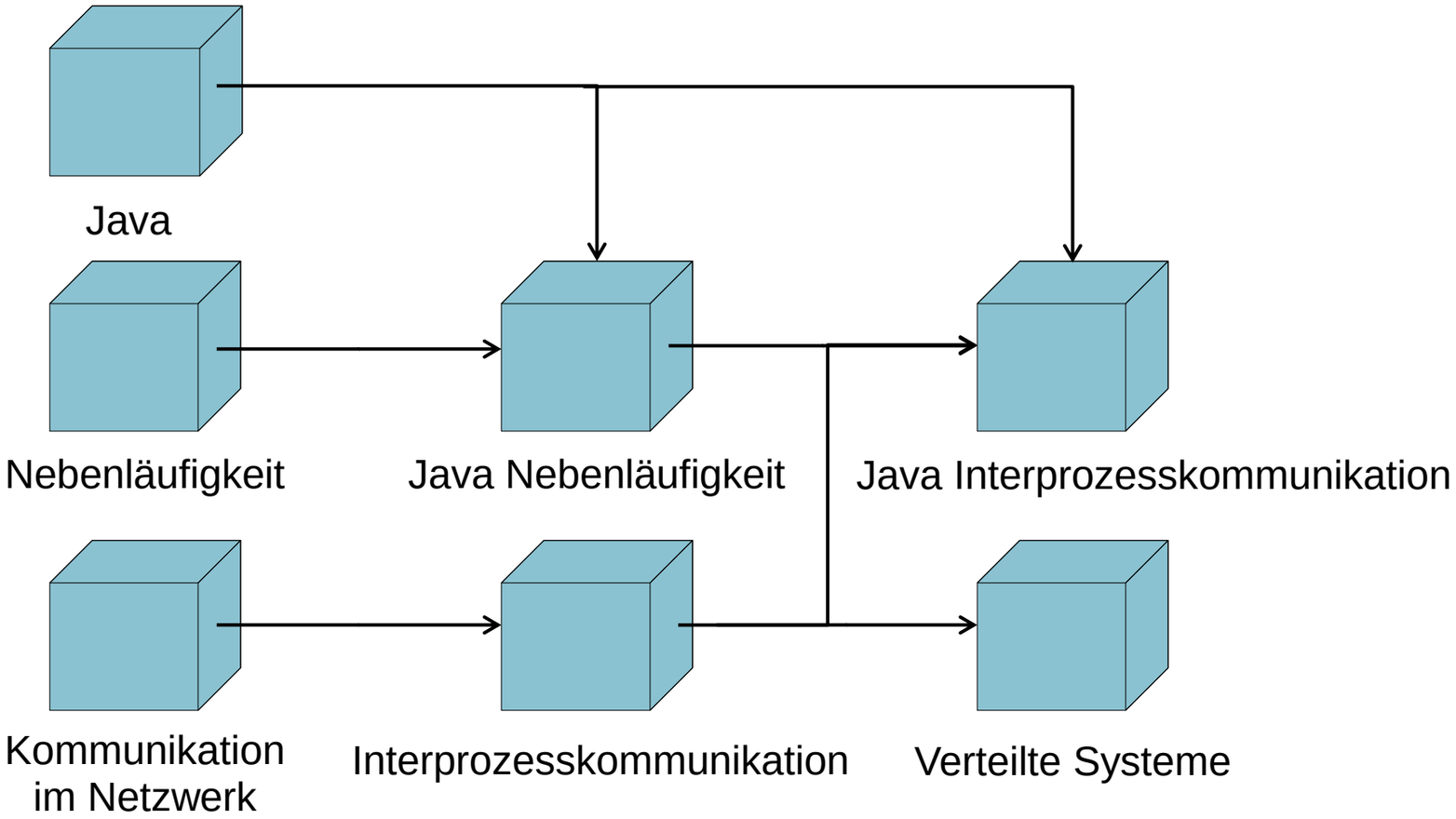

Java
Nebenläufigkeit
Java Nebenläufigkeit
Java Interprozesskommunikation
Kommunikation im Netzwerk
Interprozesskommunikation
Verteilte Systeme

ZIELE

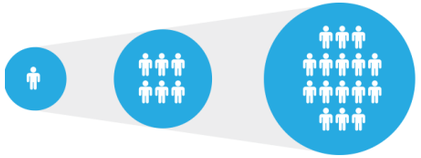
- Motivation und Herausforderungen
- Architektur
- Transparenz
- Gefahren verteilter Systeme
- Motivation Middleware

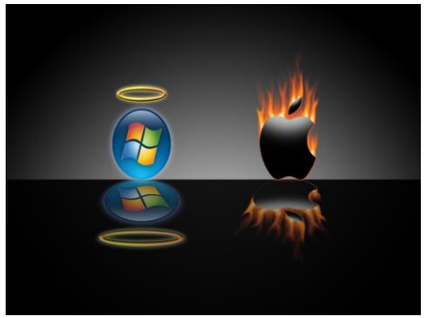
BEISPIELFRAGE

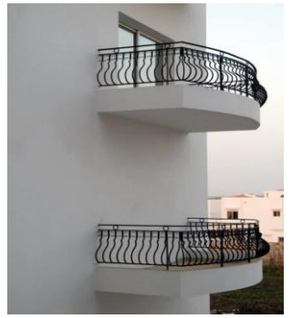
AGENDA


Java
Nebenläufigkeit
Java Nebenläufigkeit
Java Interprozesskommunikation
Kommunikation im Netzwerk
Interprozesskommunikation
Verteilte Systeme

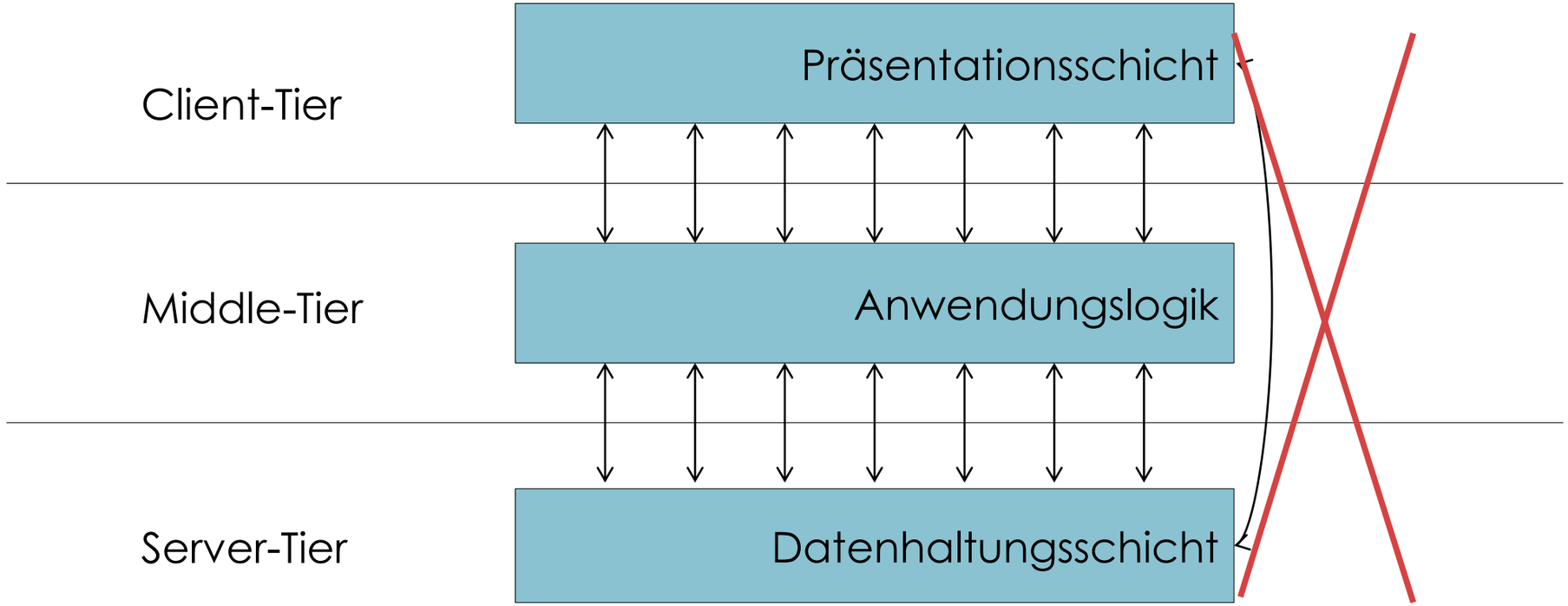
NICHT FUNKTIONALE ANFORDERUNGEN (NFA)



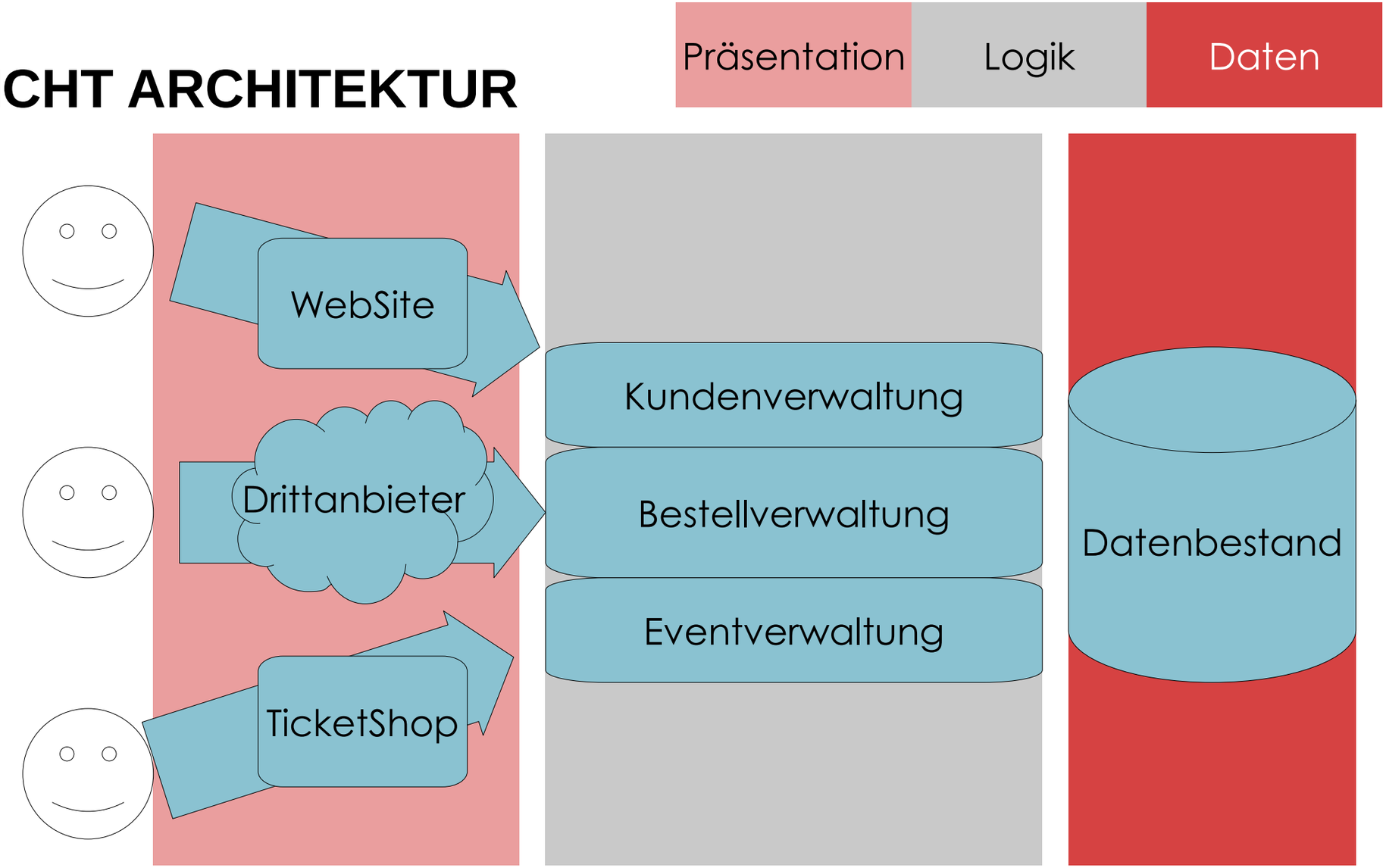




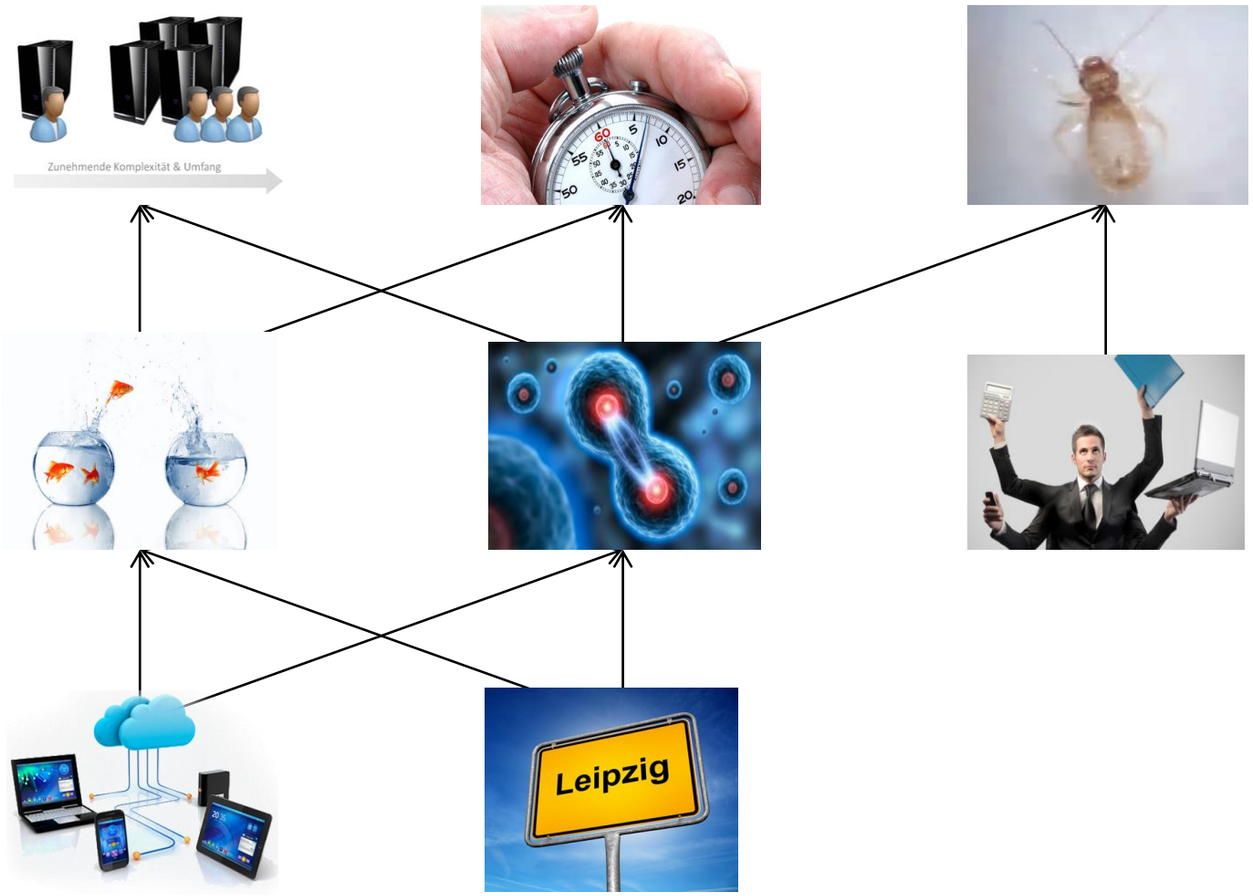
3-TIER-ARCHITEKTUR



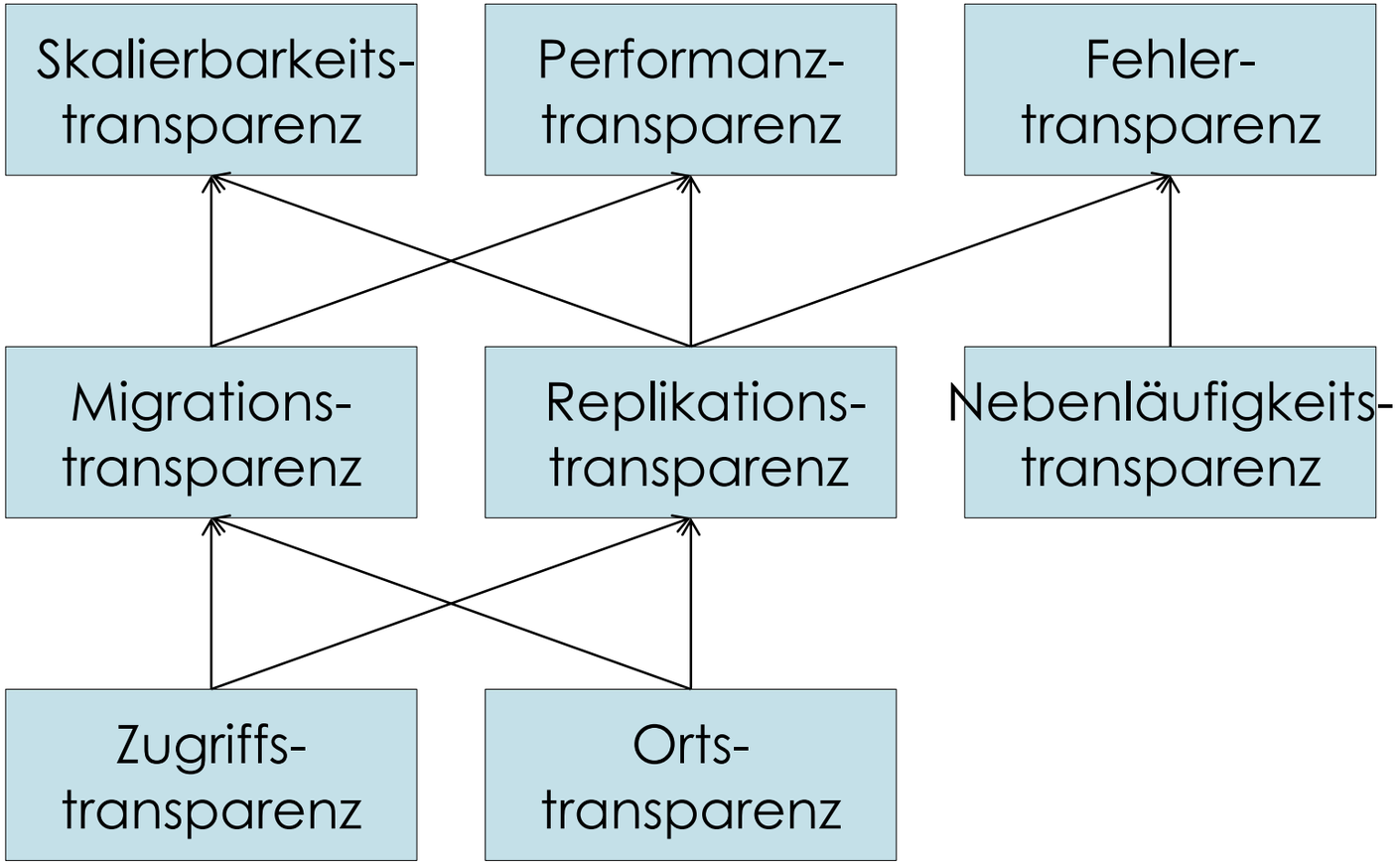
3 SCHICHT ARCHITEKTUR



DIMENSIONEN DER TRANSPARENZ

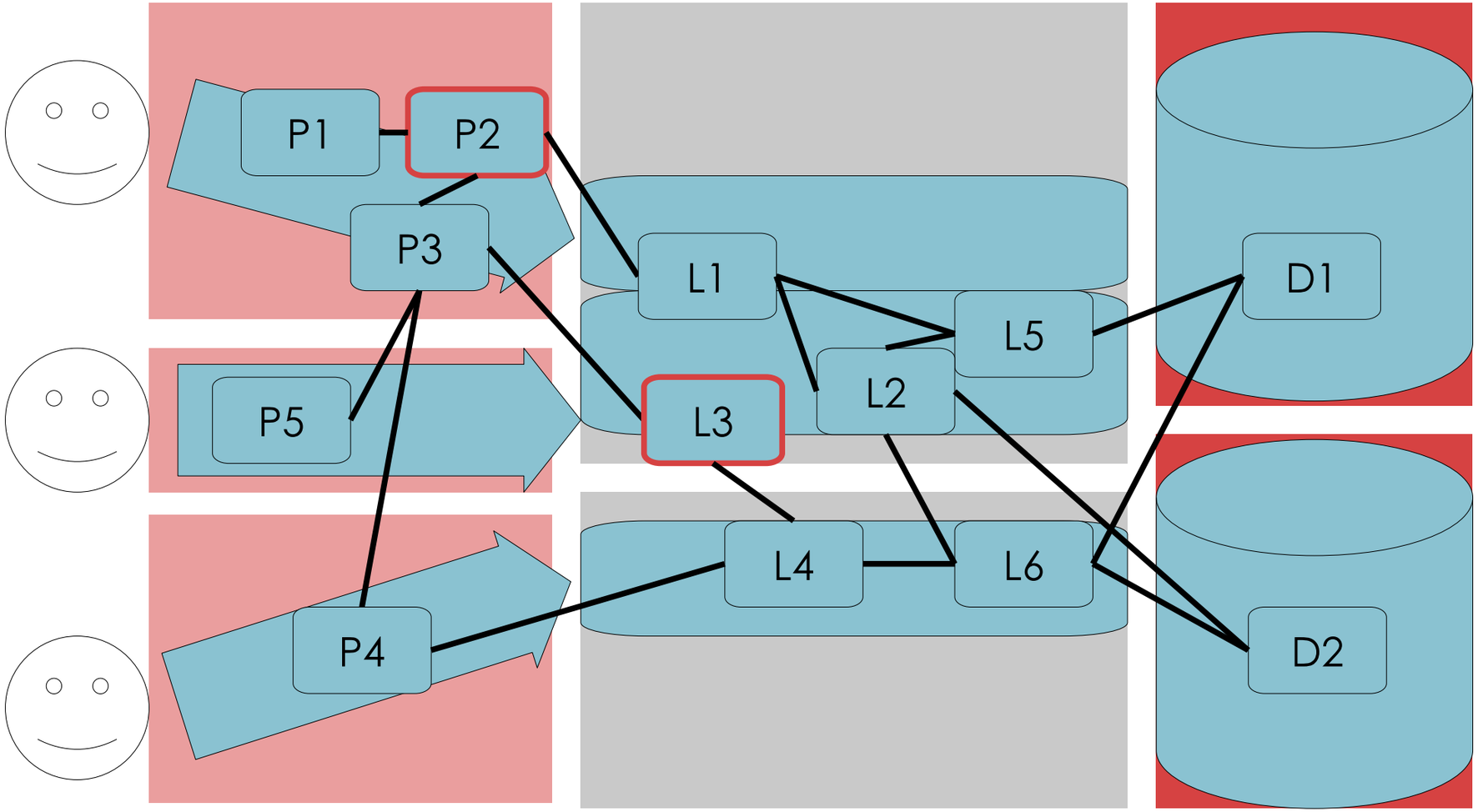


DIMENSIONEN DER TRANSPARENZ

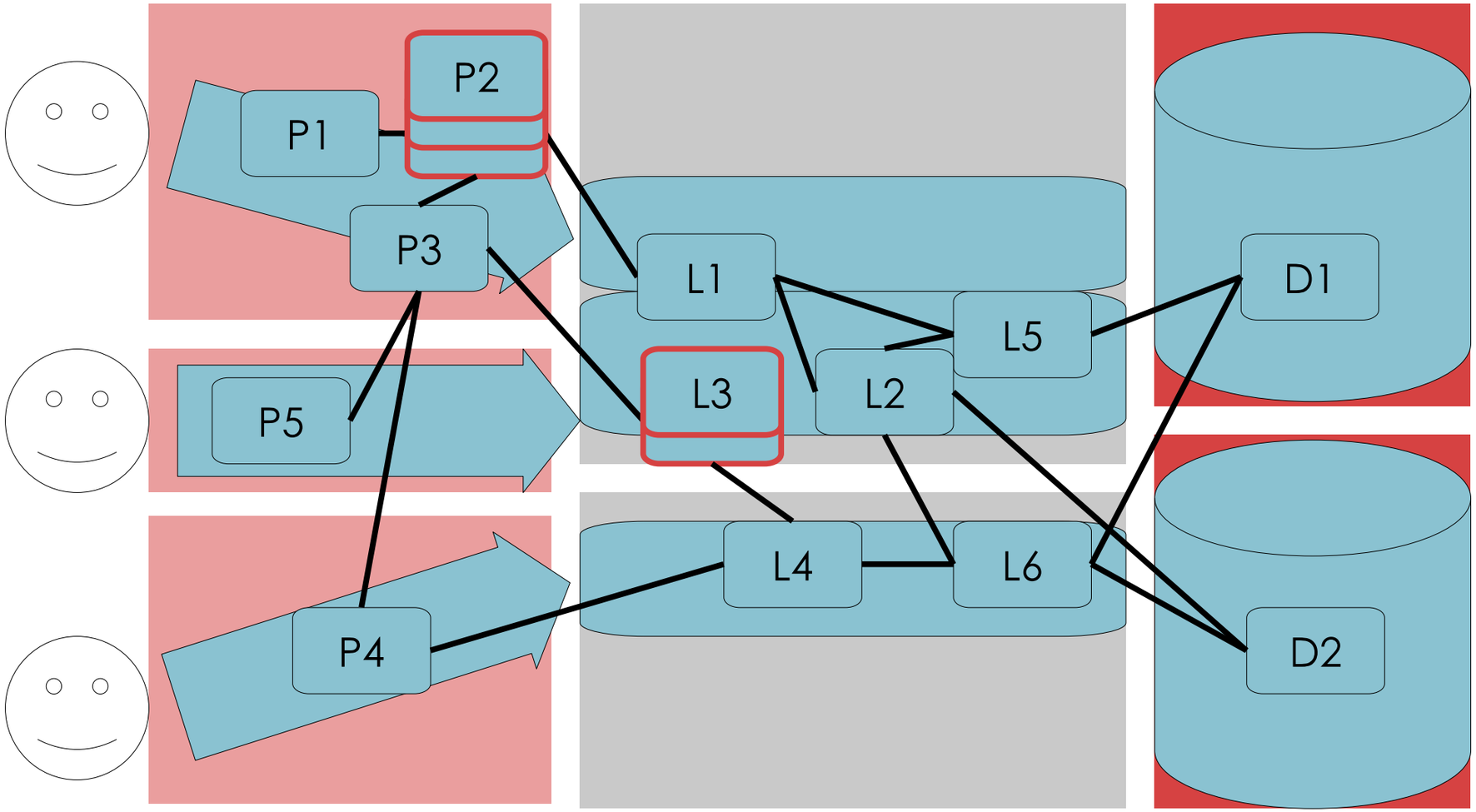


Quelle: Emmerich, W.: Konstruktion von verteilten Objekten, S. 27

REPLIKATIONSTRANSPARENZ



REPLIKATIONSTRANSPARENZ



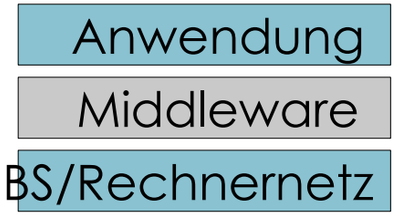
REALISIERUNG VERTEILTER KOMMUNIKATION

- Kommunikation per *Sockets* auf sehr niedrigen Abstraktionsniveau
- Eigene Protokolle für Kommunikation erzeugen Zusatzaufwand
  - Ver-/Entpackung der Byteketten für Daten, -typen und Sprachkonstrukte
  - Fallunterscheidung des zu nutzenden Dienstes
  - Aufrufe zum Senden und Empfangen der Daten
  - Überprüfung der Typsicherheit
  - Zusätzlicher überwiegend technischer Aufwand
  - Teilweise API-Untersützung beispielsweise bei Objektserialisierung

LÖSUNGSANSATZ

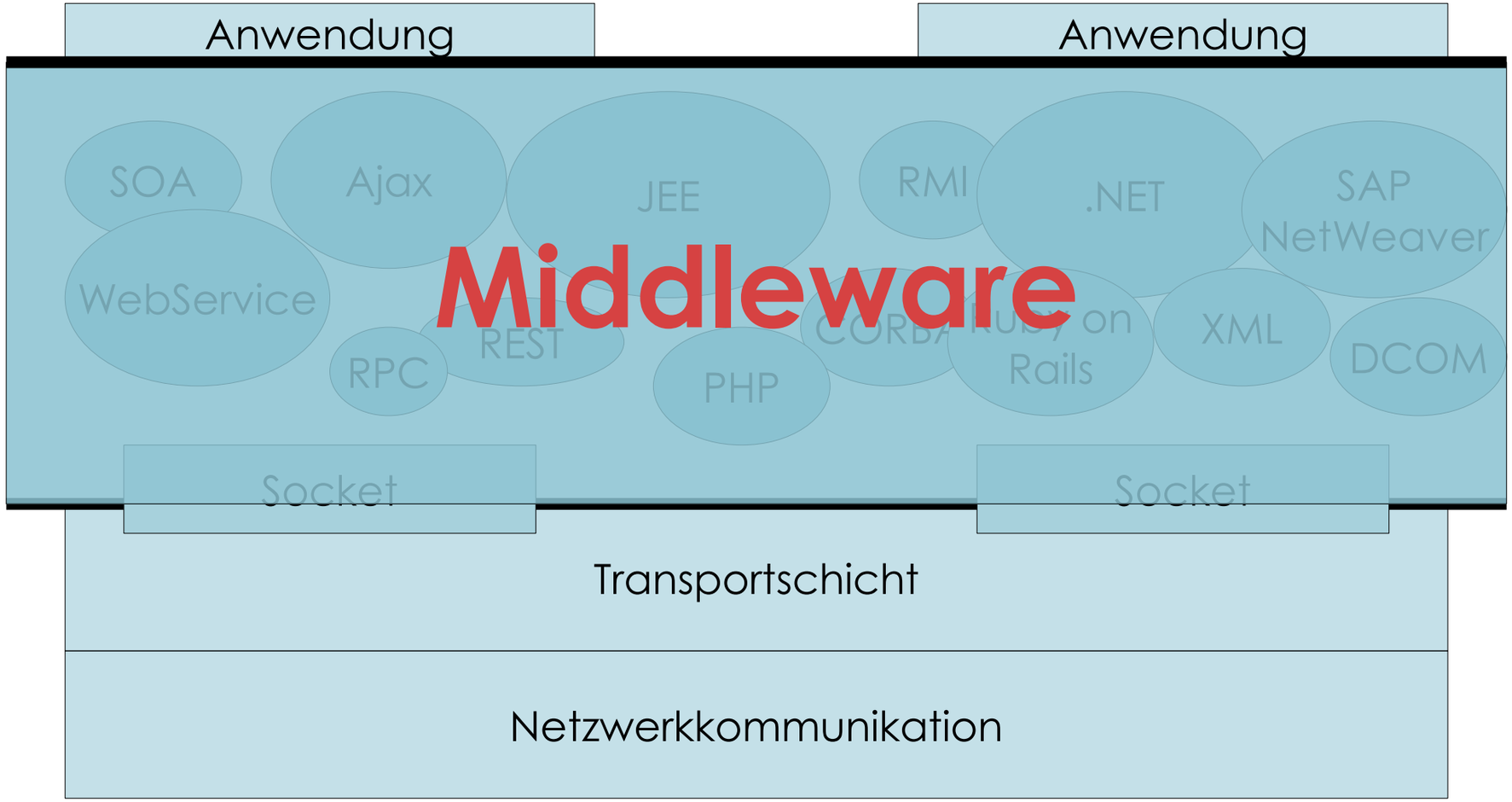
- Abstraktionsebene der Kommunikation erhöhen
- Abstrahieren von technischen Details der verteilten Kommunikation
- Konzentration auf fachliche Anforderungen
- Einführung einer anwendungsneutralen Abstraktionsebene
  - Übernimmt/Realisiert technische Aspekte der verteilten Kommunikation
  - Serialisieren, Senden, Empfangen, Deserialisieren
  - Identifikation, Aufrufweiterleitung, Fehlerbehandlung
- Realisiert Transparenz für den Nutzer und den Entwickler!

MIDDLEWARE



- „Zwischenanwendung“ oder anwendungsneutrale Programme
- Vermitteln zwischen Anwendung und Infrastruktur
- Komplexität des Gesamtsystem und der Infrastruktur bleibt verborgen
- Eine Ebene in einem komplexen Softwaresystem
- Implementiert die benötigten Transparenzdimensionen
- Realisiert somit die NFA (Skalierbarkeit, Wartbarkeit, Performanz,...)
- Ermöglicht entkoppelten Softwarekomponenten den Datenaustausch

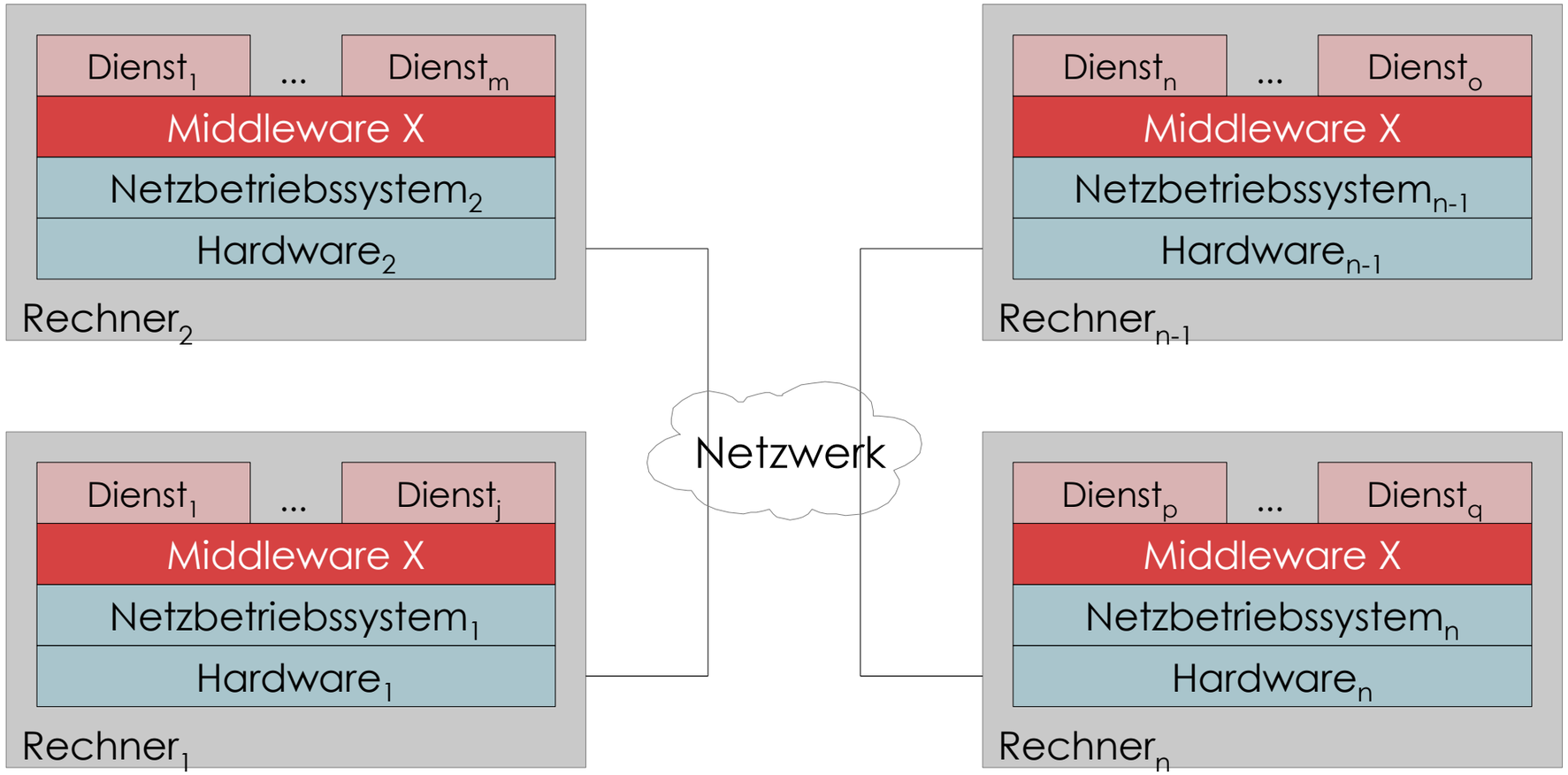
MIDDLEWARE


Anwendung
Anwendung
SOA
Ajax
JEE
RMI
.NET
SAP NetWeaver
WebService
Middleware
REST
RPC
PHP
CORBA
Ruby on Rails
XML
DCOM
Socket
Socket
Transportschicht
Netzwerkkommunikation

MOTIVATION FÜR MIDDLEWARE

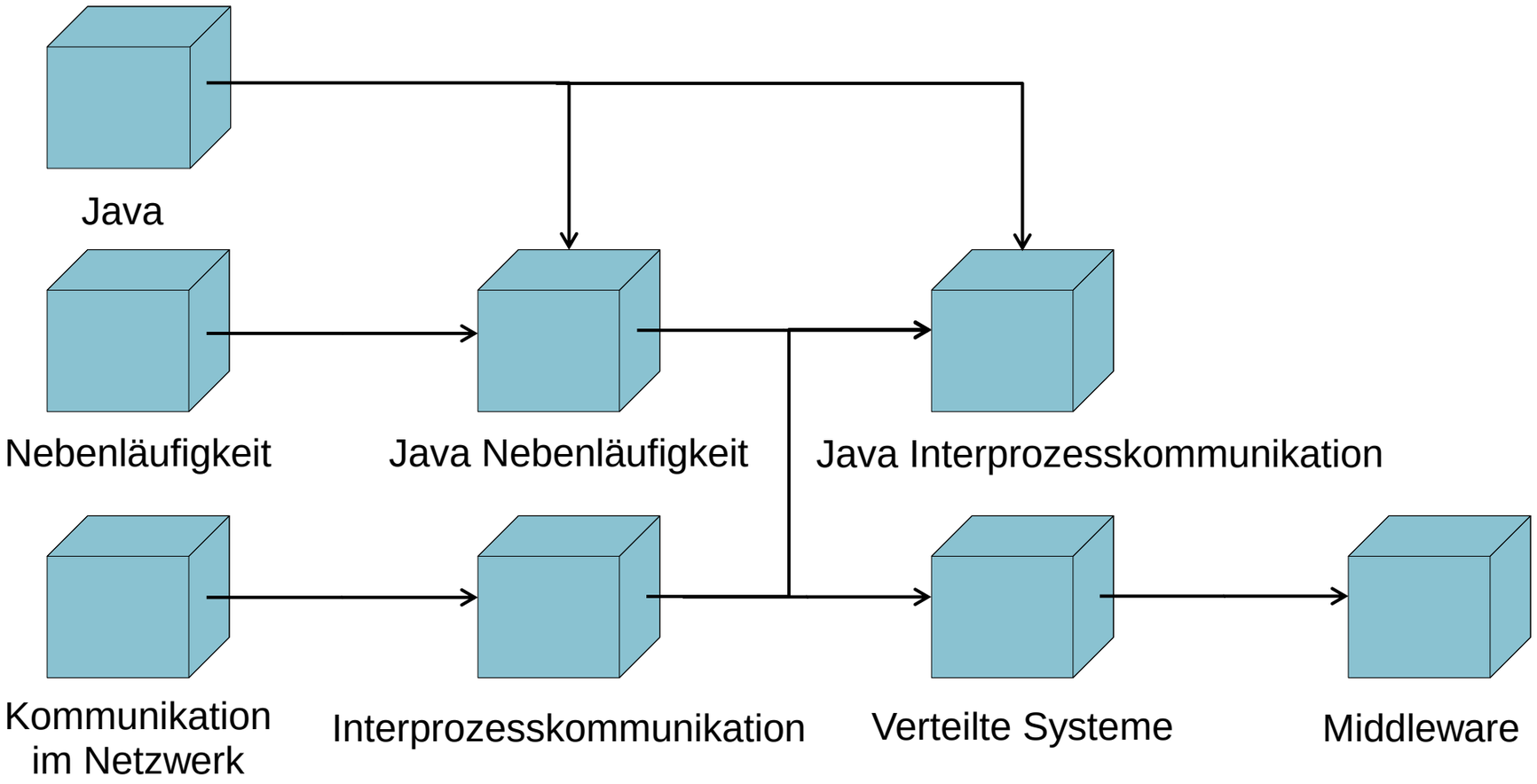
- NFA und Transparenz bedingen **verteilte** komplexe Kommunikation
- Handling der Kommunikation sehr aufwändig und fehleranfällig
- Zusätzliche Herausforderung durch Heterogenität der Systeme
  - Hardware
  - Betriebssystem
  - Programmiersprache
  - Versionen
- Middleware als einheitliche Plattform für Entwickler verteilter Systeme
  - Auslagerung technischer Details der Kommunikation
  - Konzentration auf Realisierung der fachlichen Anwendung
  - Verfügbarkeit ergänzender Dienste (Naming, Security und Transaction)
- Idealvorstellung: Baukasten lose gekoppelter interoperabler Dienste

MODELL EINES VERTEILTEN SYSTEMS



Ähnlich zu: Emmerich, W.: Konstruktion von verteilten Objekten, S. 9

AGENDA


Java
Nebenläufigkeit
Java Nebenläufigkeit
Java Interprozesskommunikation
Kommunikation im Netzwerk
Interprozesskommunikation
Verteilte Systeme
Middleware

ZIELE MIDDLEWARE

- Mehrwert kommunikationsorientierter Middleware kennen
- 3 Modelle der kommunikationsorientierten Middleware kennen
- Mehrwert anwendungsorientierter Middleware verstehen
- Dienste von anwendungsorientierter Middleware kennen
- Komponentenparadigma verstehen

ÜBERSICHT

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware

ZIELE KOMMUNIKATIONSORIENTER MIDDLEWARE

- Abstraktion von der Netzwerkprogrammierung (Sockets)
- Prozeduren/Funktionen/Methoden – Aufruf statt Strömen
- Unterstützung Datenübertragung zwischen entfernten Komponenten
- Erweiterung der Netzverbindung durch Dienste
  - Datenanpassungen: Marshalling/Unmarshalling, Formatekonvertierung
  - Fehlerbehandlung : Prüfsummen, Ausfallbehandlung, Sendewiederholung
- Realisierung durch Middlewareprotokoll
- Setzt auf Protokollen IP, TCP/UDP auf

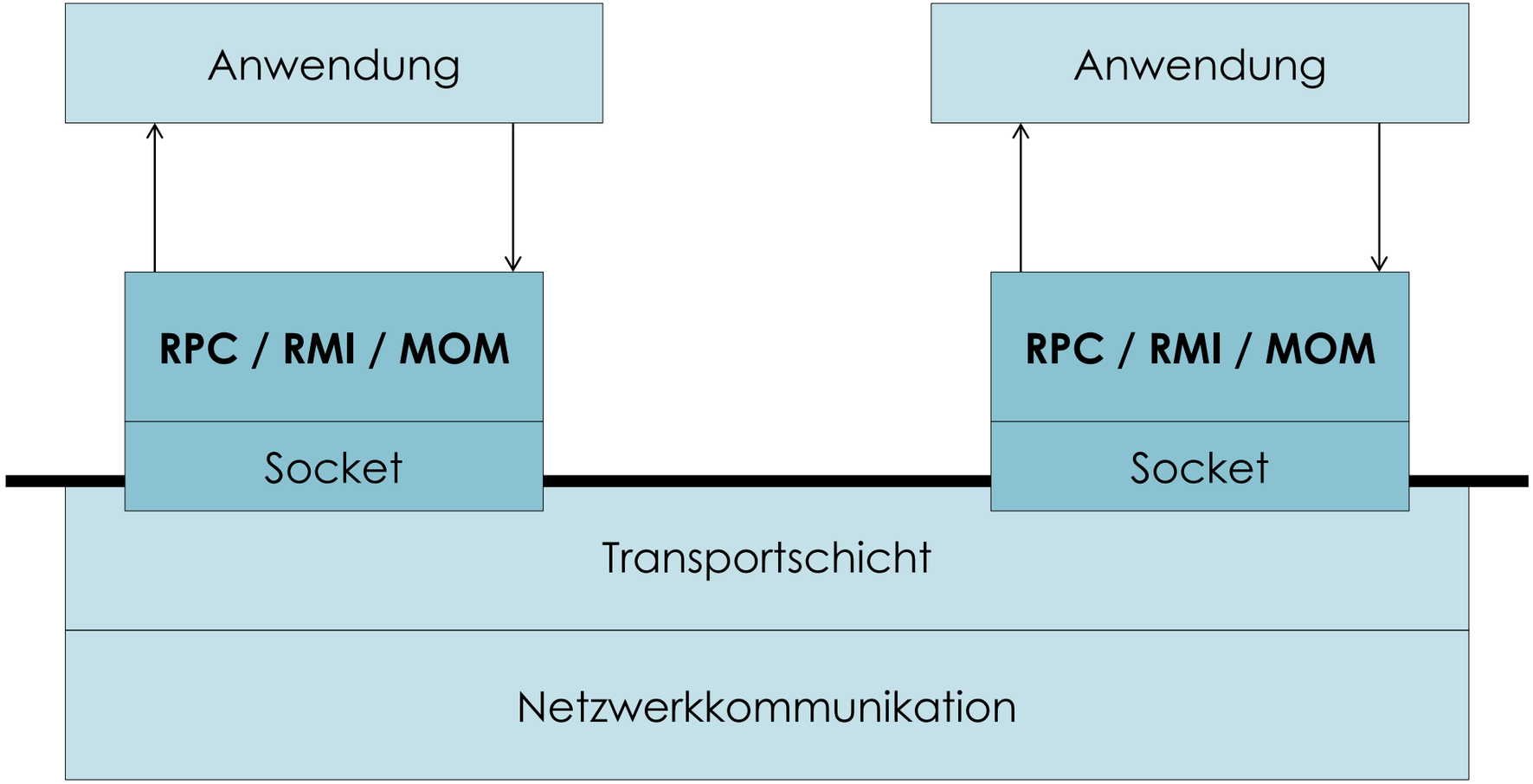
DATENTRANSFORMATION & HETEROGENITÄT

- Datentransformation
  - (Un)Marshalling der Aufrufparameter in übertragungsfähiges Format
  - Sicherung eines einheitlich definierten Datentyps für Sender & Empfänger
- Heterogenität Hardware und Betriebssystem
  - Ausgleichen der Heterogenität verschiedener OS/HW Kombinationen
  - Zulassen Individualität der einzelnen Knoten (z.B. Endian-Darstellung)
  - Übertragung in vereinbarter Network Byte Order
  - Unterschiedliche Formate der Zeichendarstellung in Betriebssystemen (IBM-Mainframes: EBCDIC-Format, Windows, UNIX – ASCII-Code)

PROGRAMMIERMODELLE

- 1) Prozedurales Modell
  - Entfernter Prozeduraufruf (Remote Procedure Call RPC)
  - Synchrone Kommunikation
- 2) Objektorientiertes Modell
  - Entfernter Methodenaufruf (Remote Method Invocation RMI)
  - Synchrone Kommunikation
- 3) Nachrichtenbasiertes Modell
  - Nachrichtenversand (Message oriented Middleware MOM)
  - Verschicken/Empfangen von Nachrichten
  - Asynchrone Kommunikation über Warteschlangen

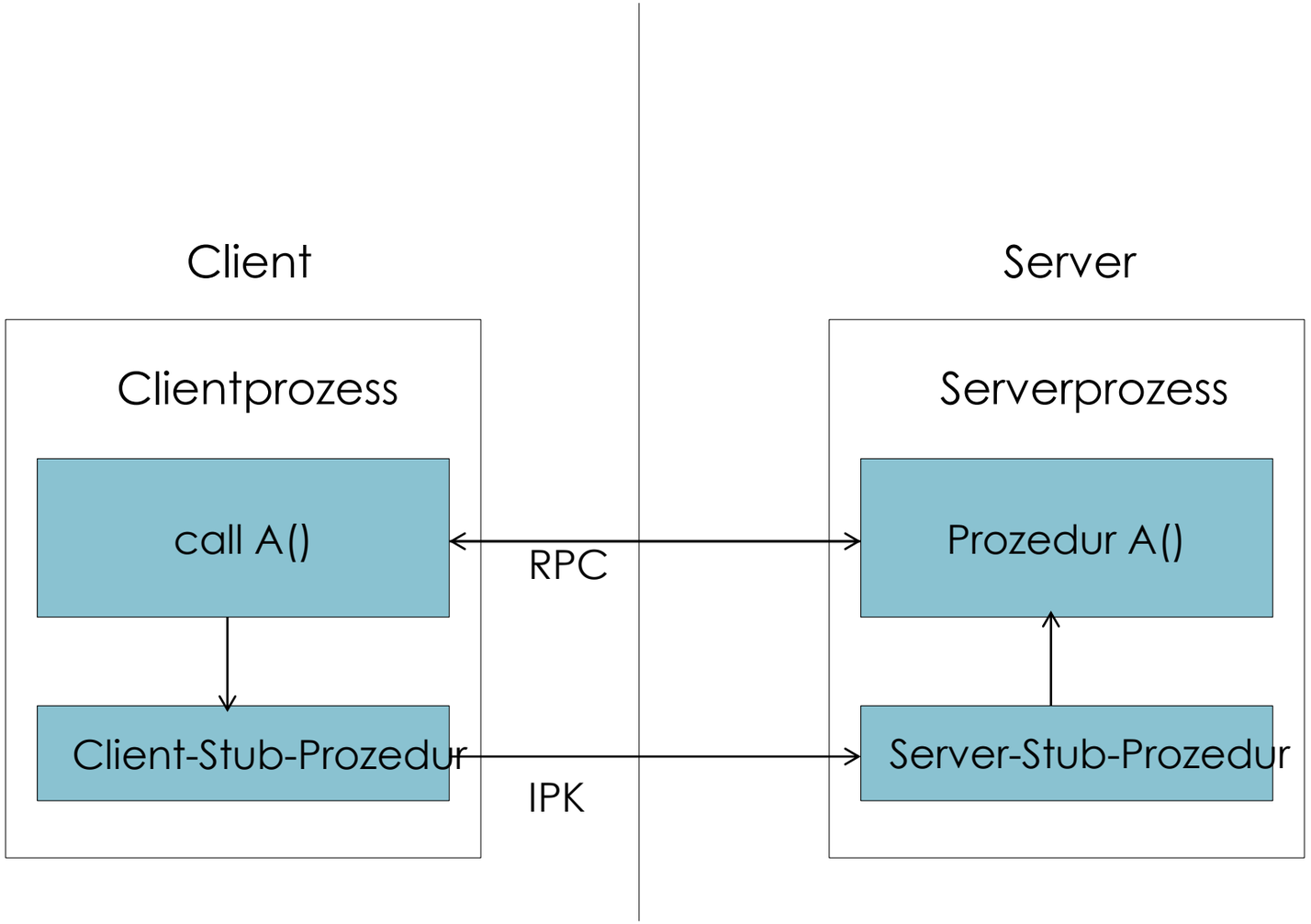
RFC/RMI/MOM UND SOCKETS



ÜBERSICHT

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware

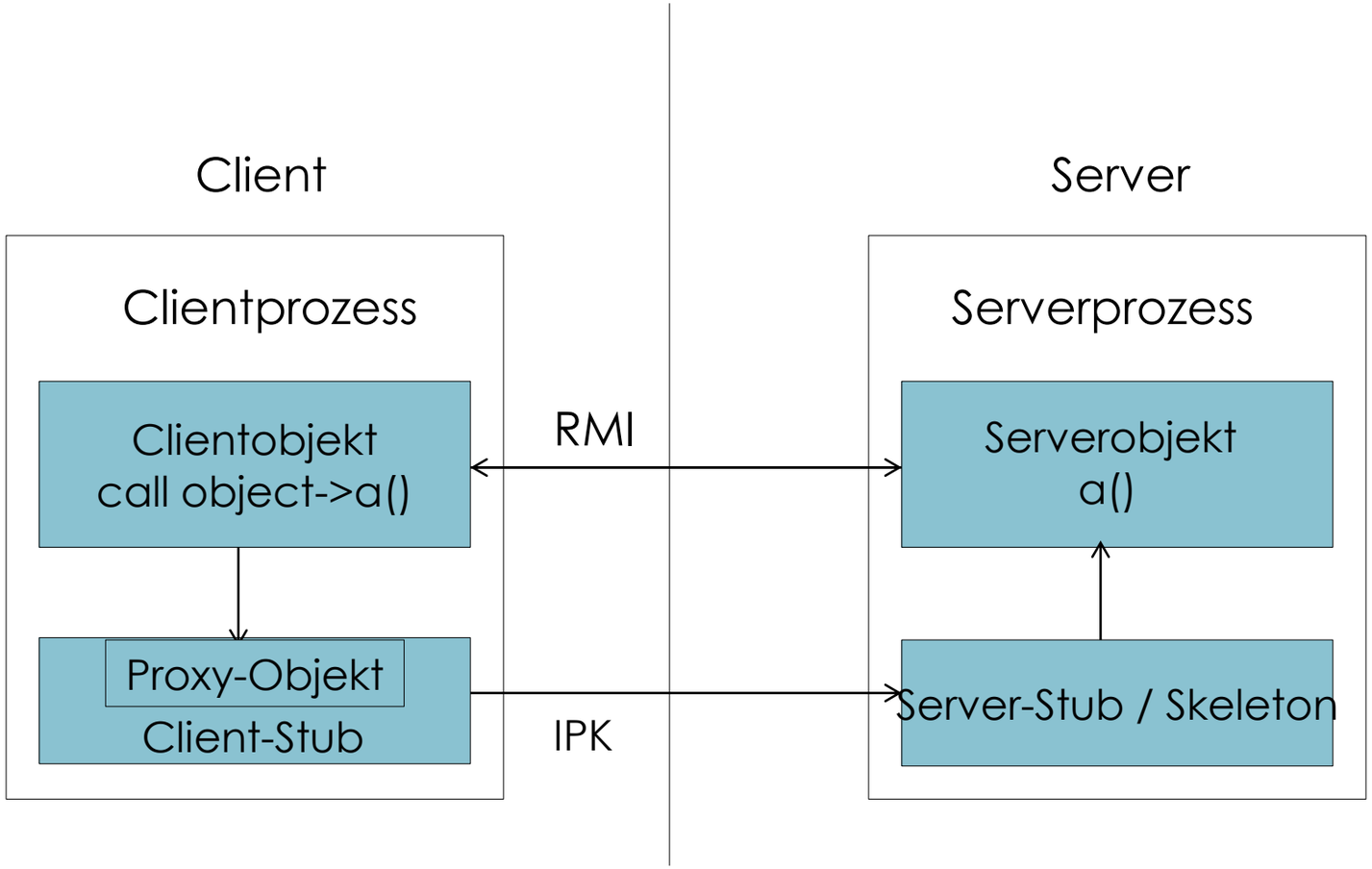
1) REMOTE PROCEDURE CALL (RPC)



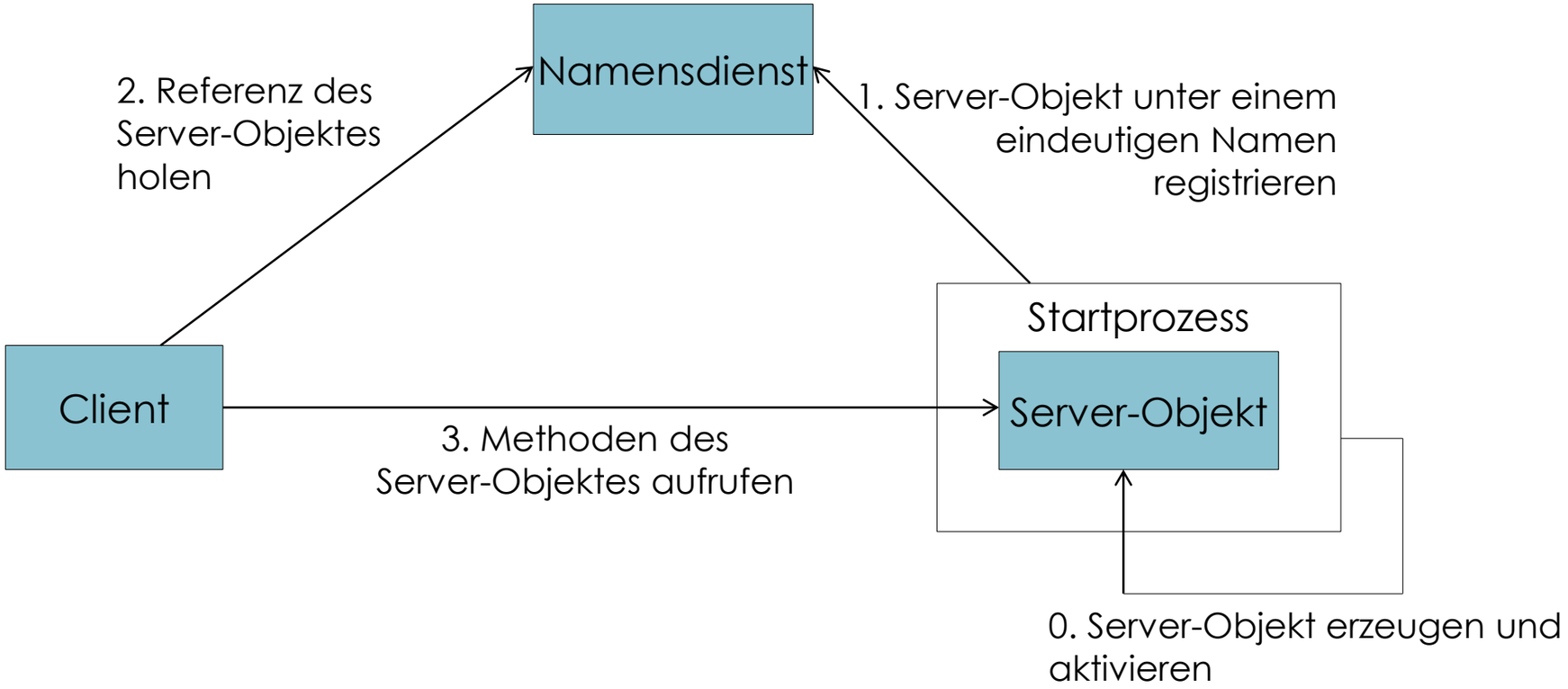
ÜBERSICHT

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware

REMOTE METHOD INVOCATION (RMI)



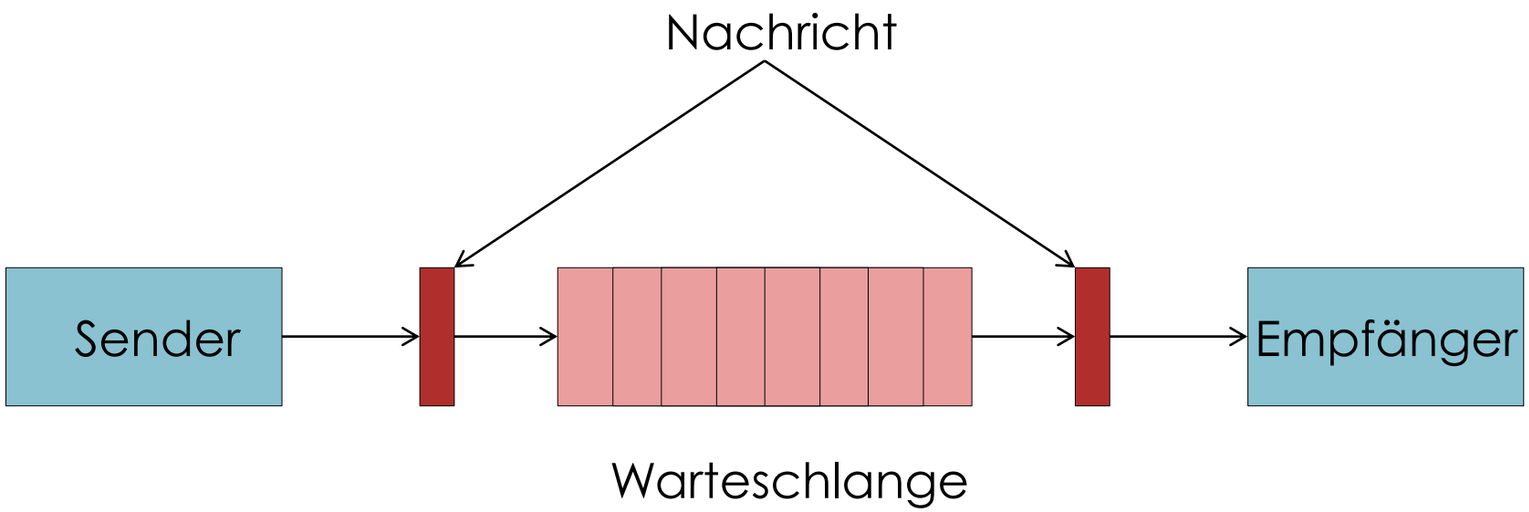
AUFBAU RMI


Namensdienst
2. Referenz des Server-Objektes holen
1. Server-Objekt unter einem eindeutigen Namen registrieren
Startprozess
Client
Server-Objekt
3. Methoden des Server-Objektes aufrufen
0. Server-Objekt erzeugen und aktivieren

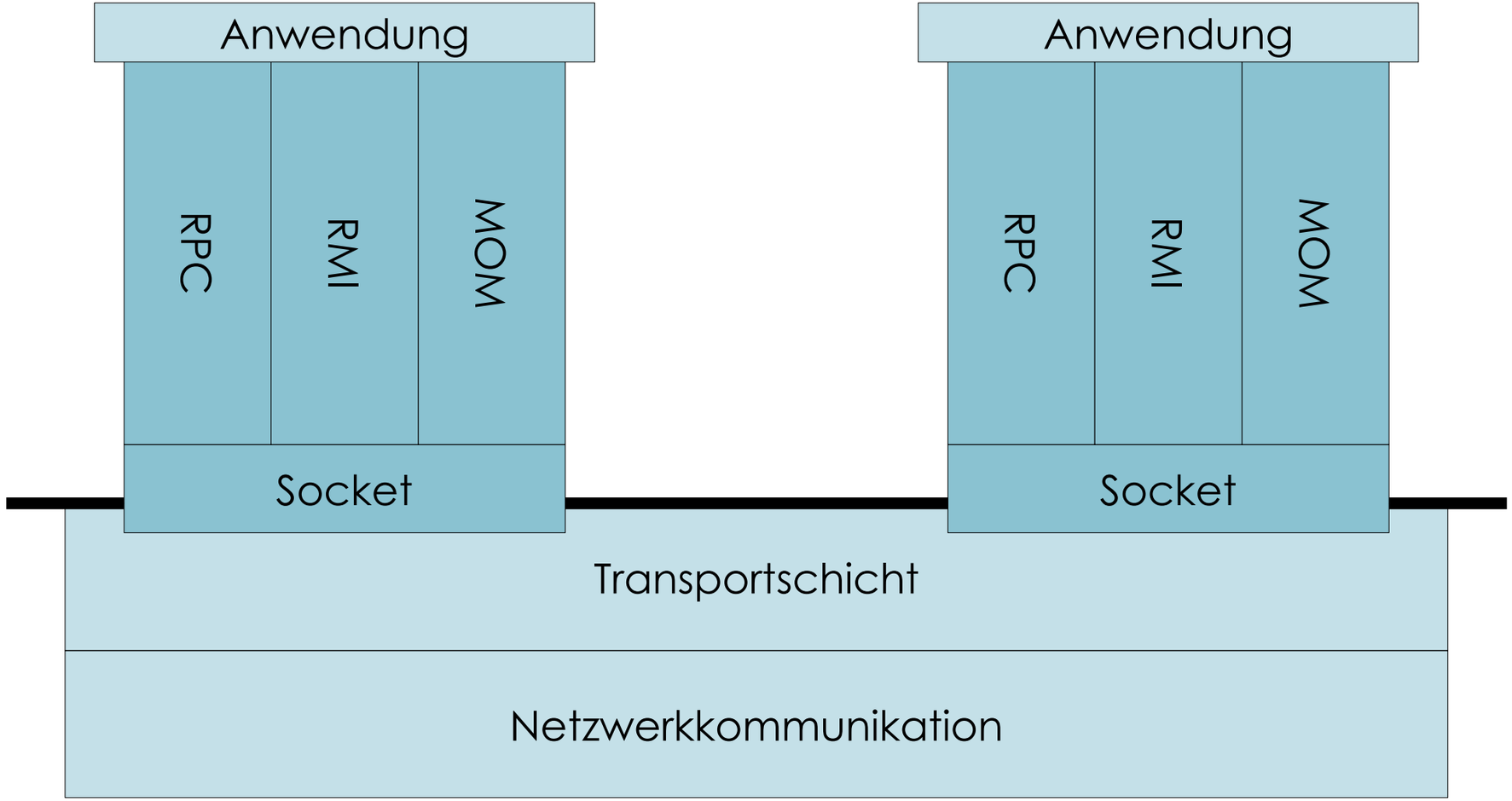
ÜBERSICHT

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware

MESSAGE ORIENTED MIDDLEWARE (MOM) BEISPIEL LOGGING UND PERFORMANZEINFLUSS



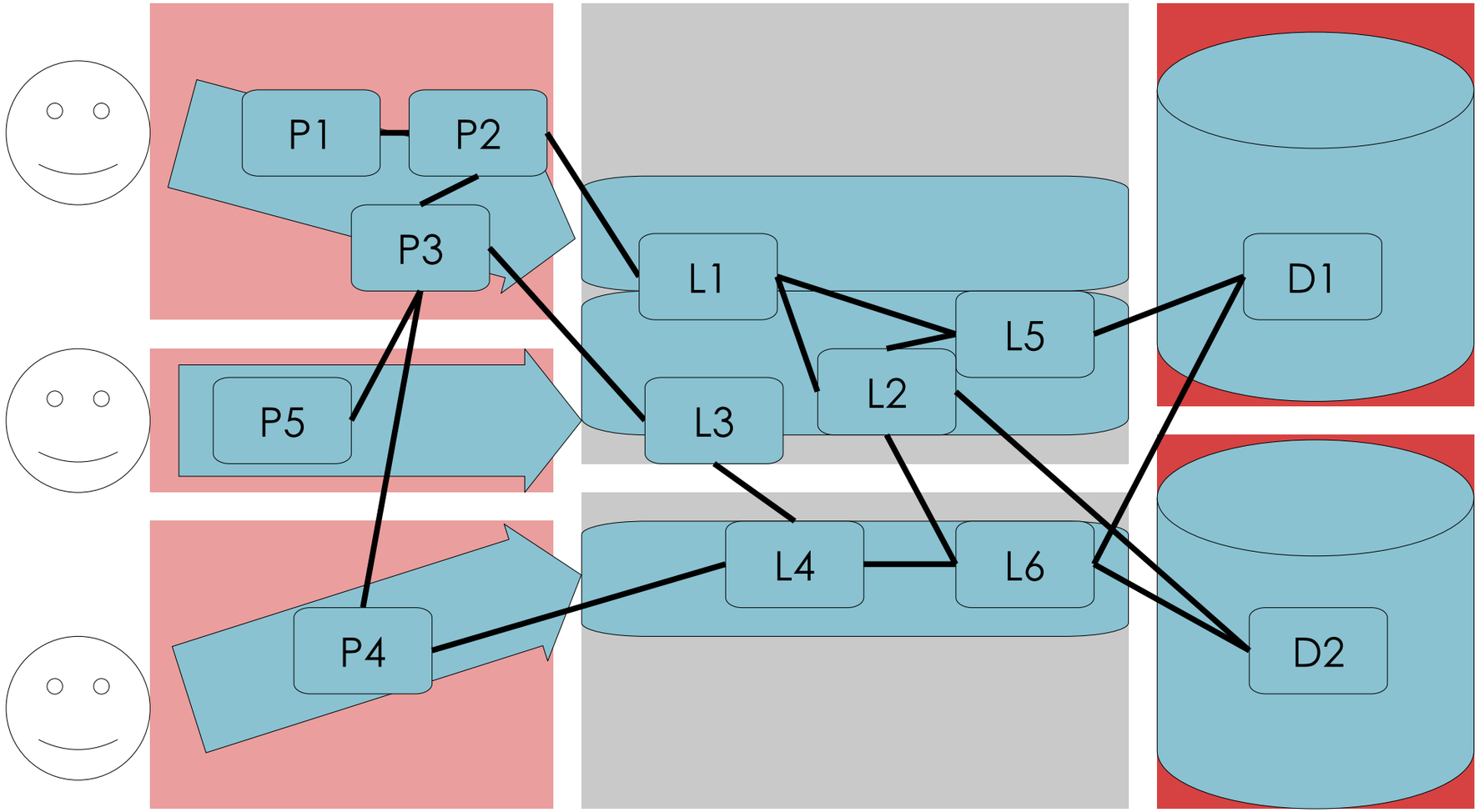
KOMMUNIKATIONSORIENTIERTE MIDDLEWARE



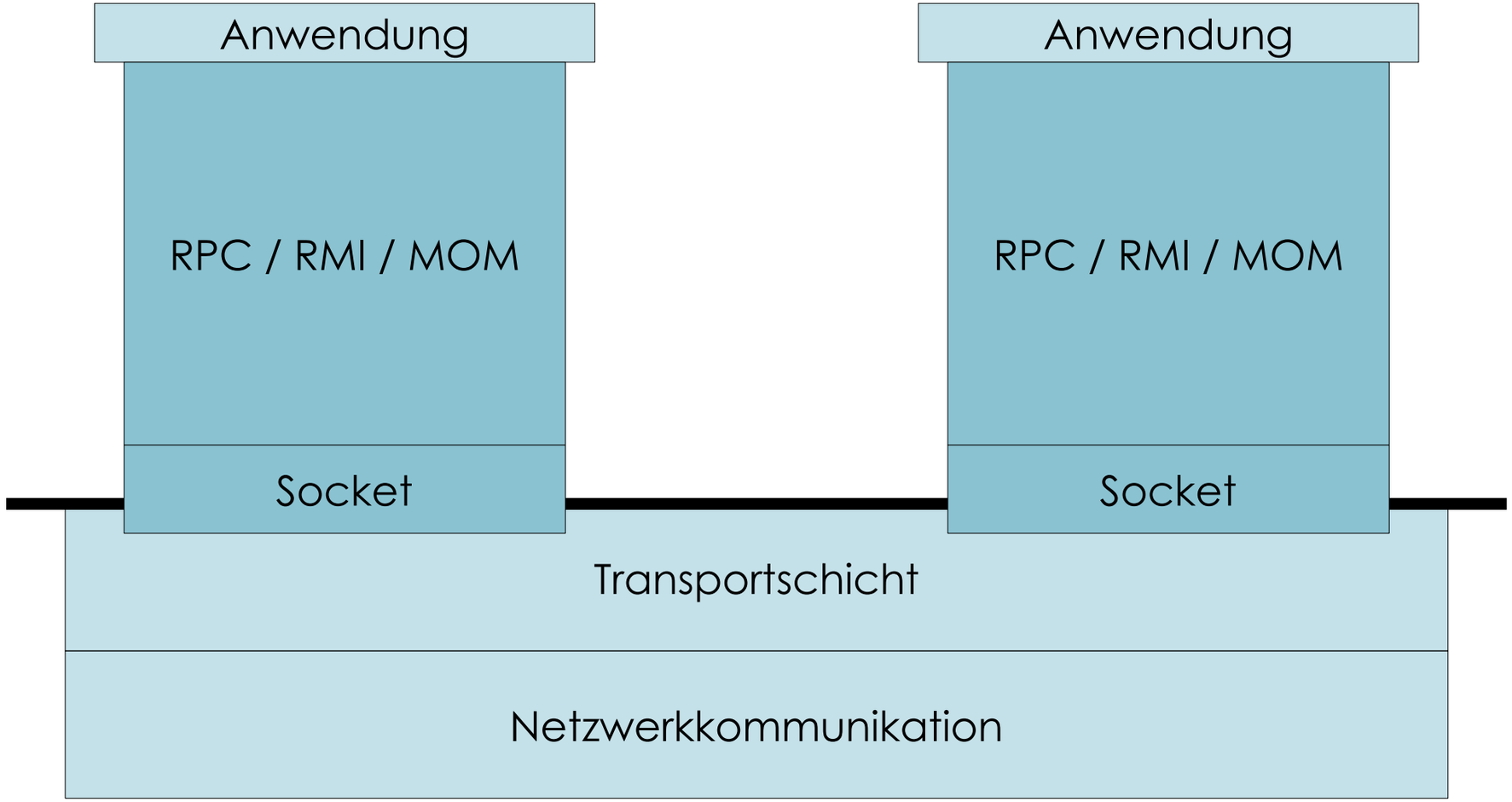
ÜBERSICHT

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware
  - Dienste
  - Komponentenmodell
  - Component Object Model (COM)
  - CORBA

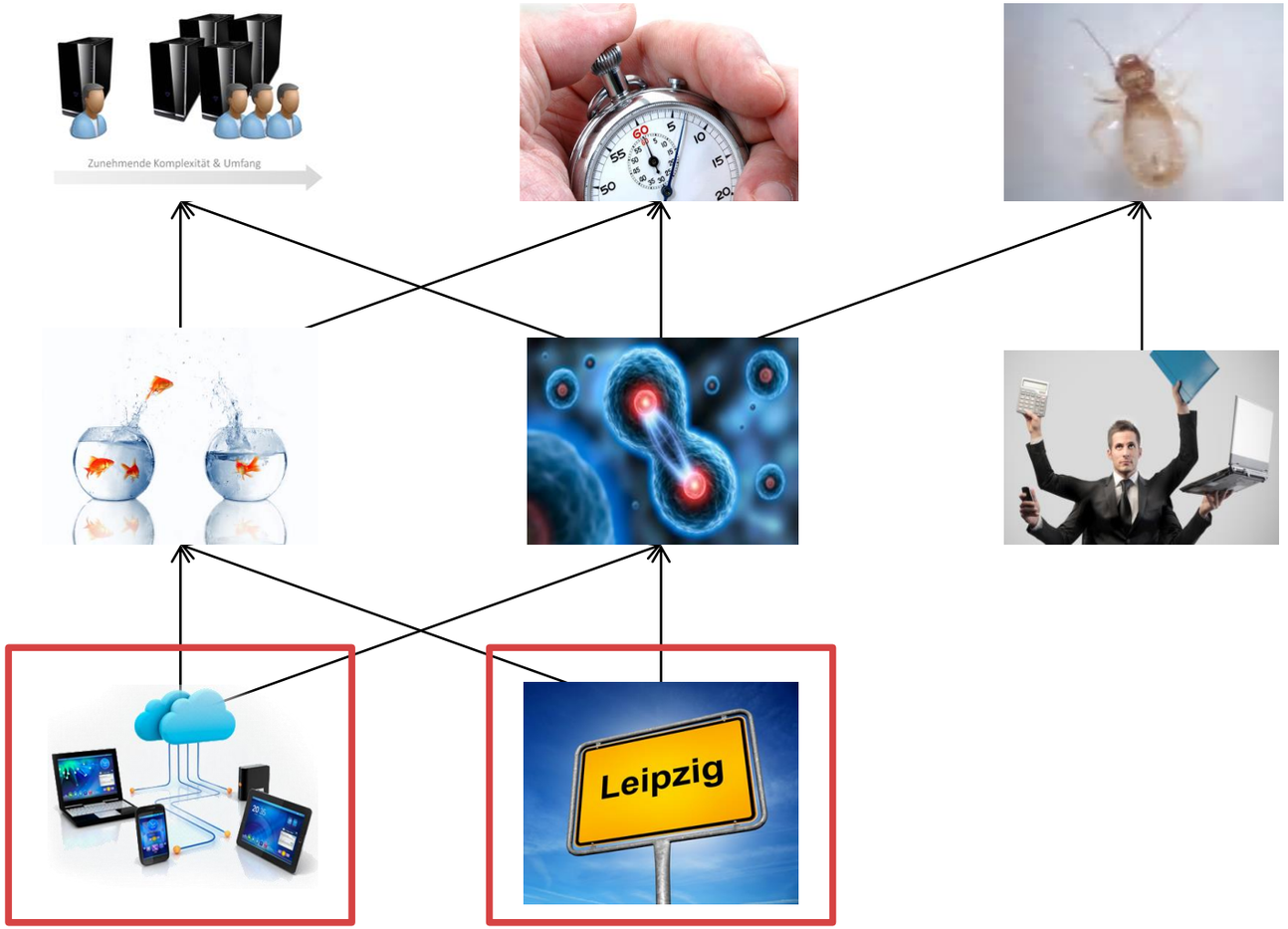
VERTEILUNG AUF KOMPONENTEN UND RECHNER



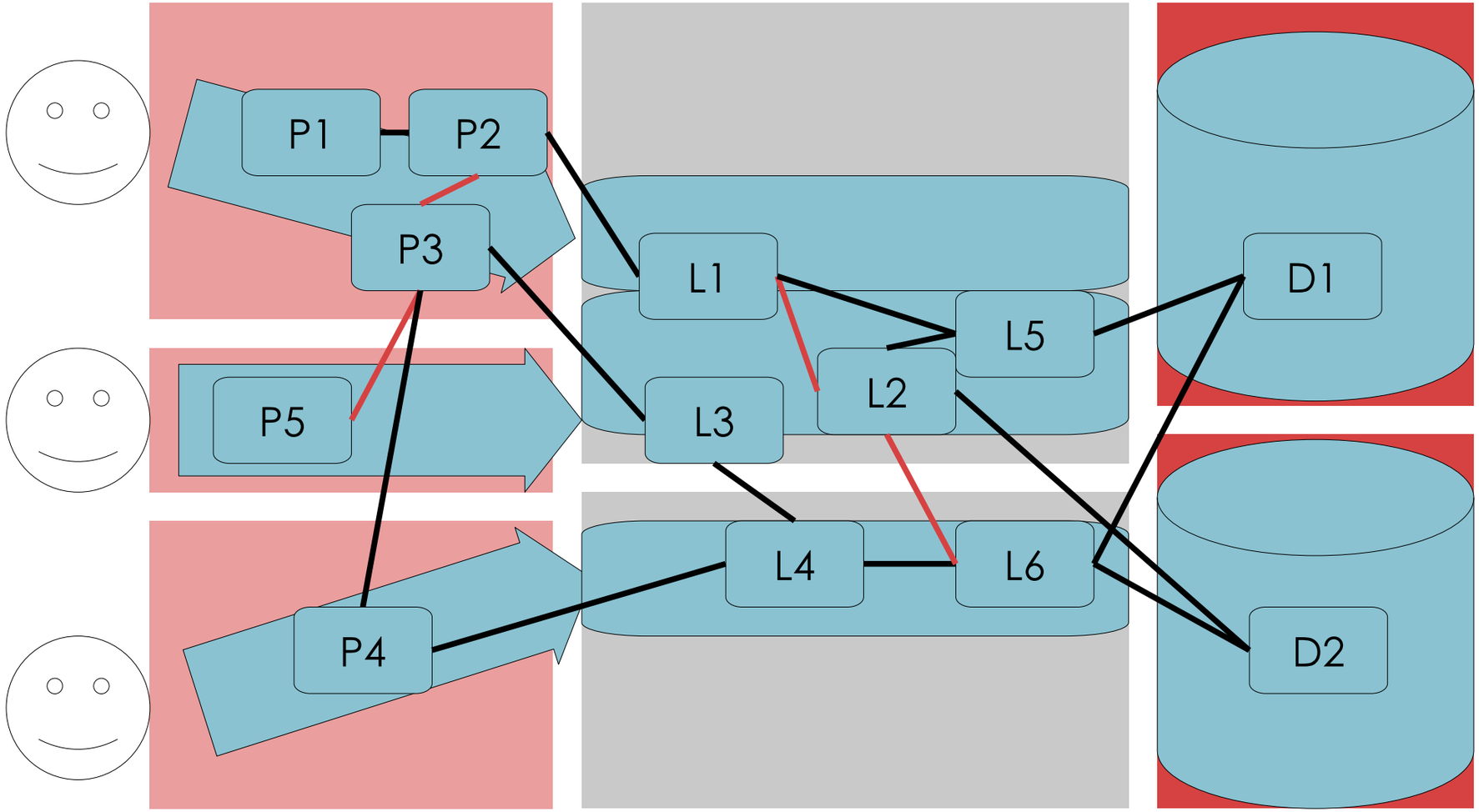
KOMMUNIKATIONSORIENTIERTE MIDDLEWARE



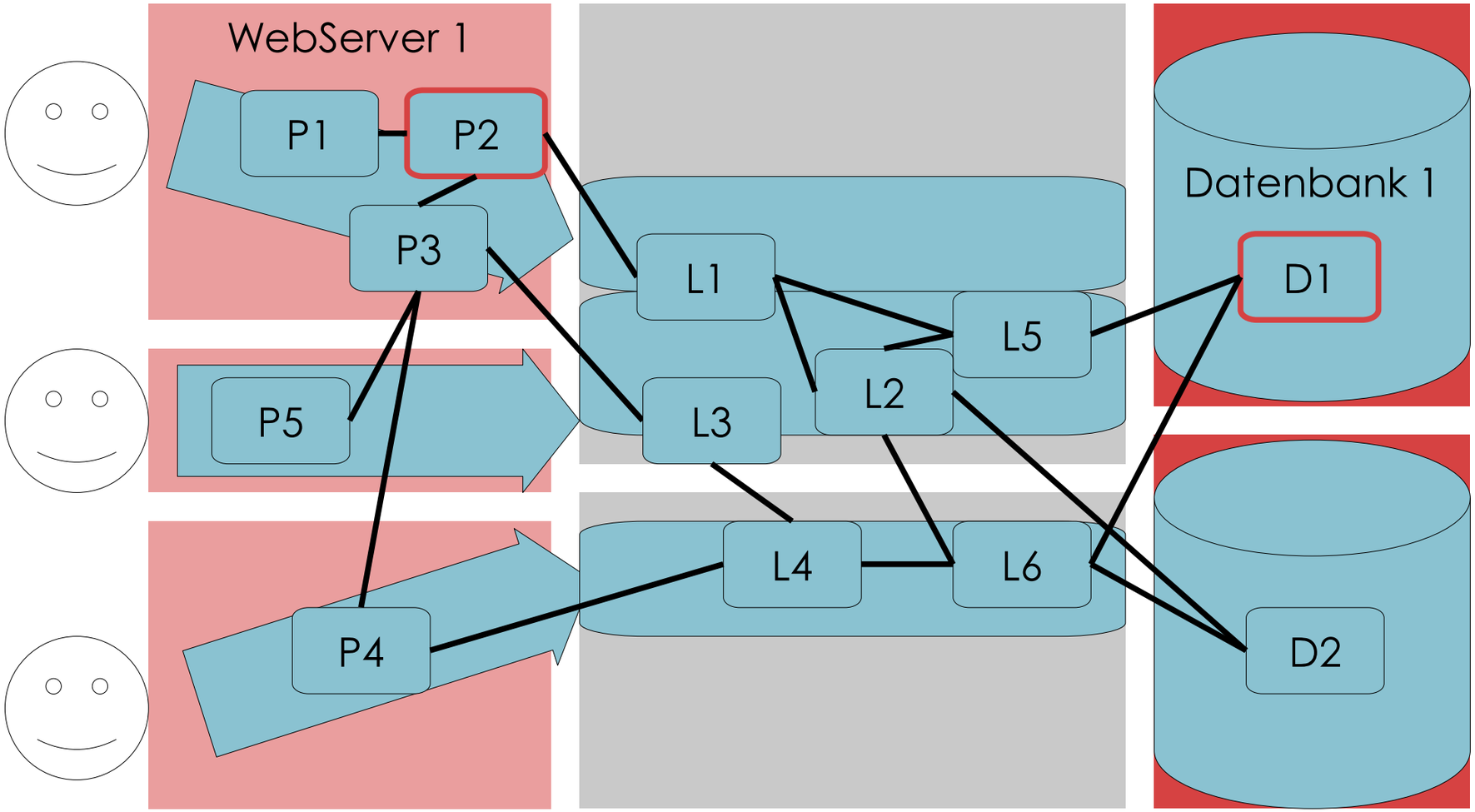
TRANSPARENZ DAMIT REALISIERBAR...



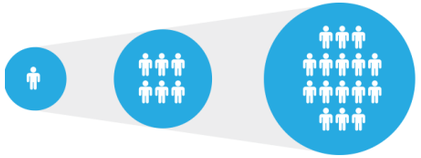
ZUGRIFFSTRANSPARENZ

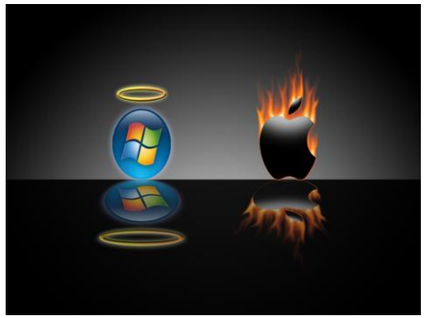


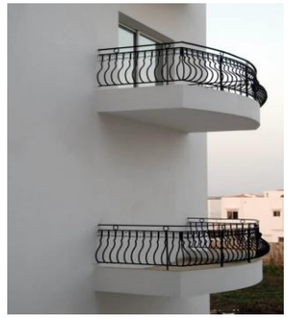
ORTSTRANSPARENZ



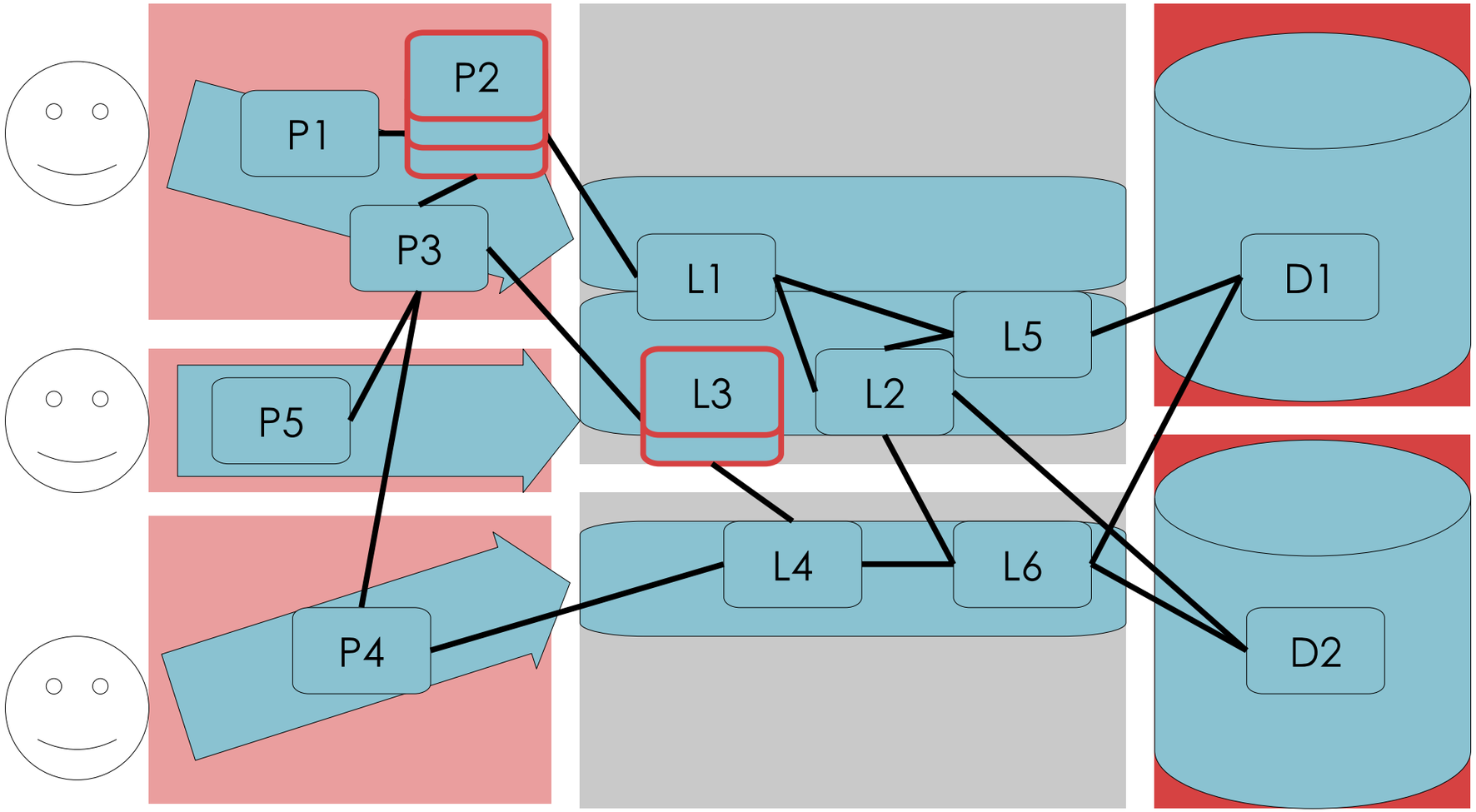
NFA DAMIT REALISIERBAR







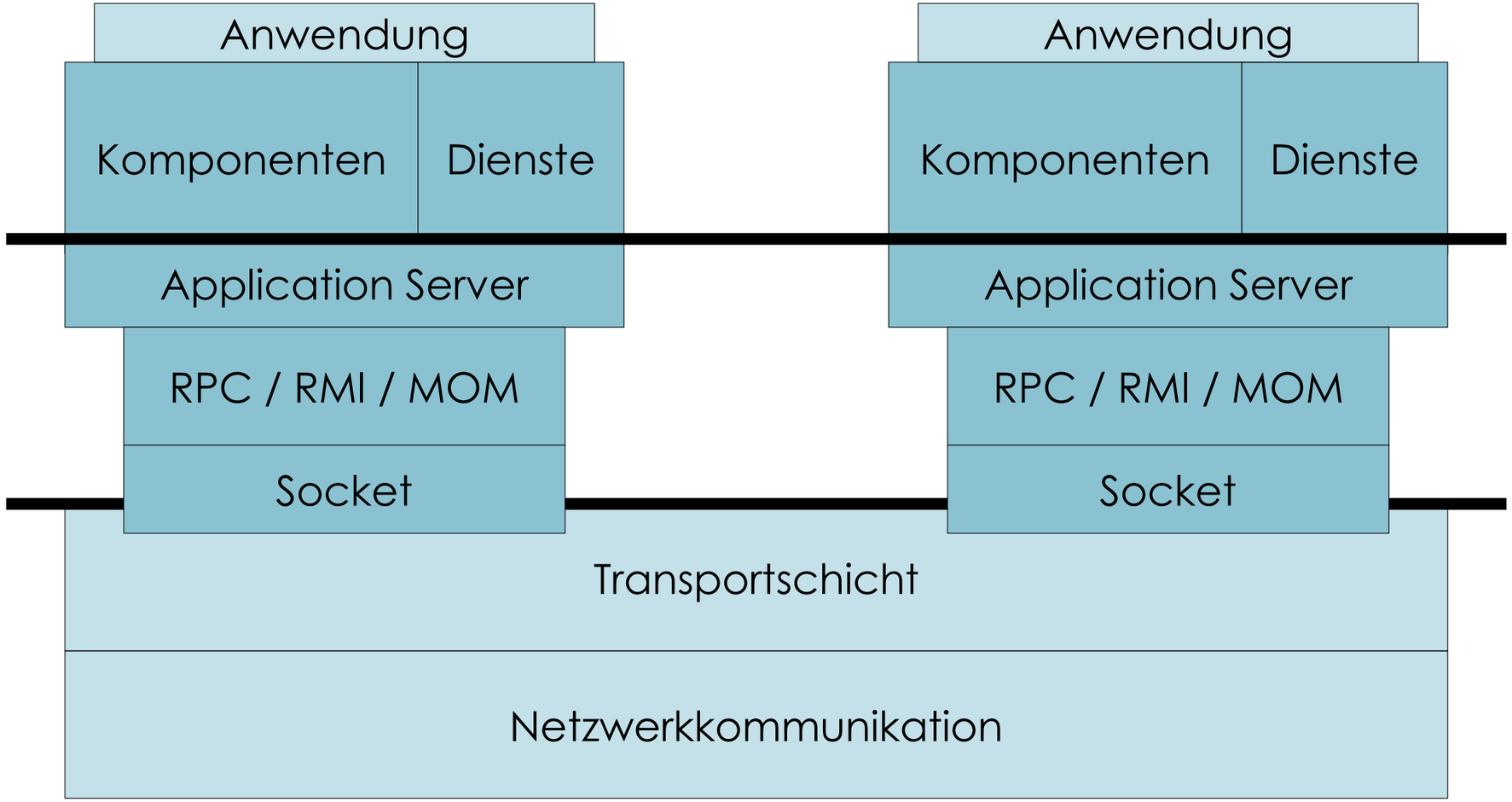
REPLIKATIONSTRANSPARENZ ???



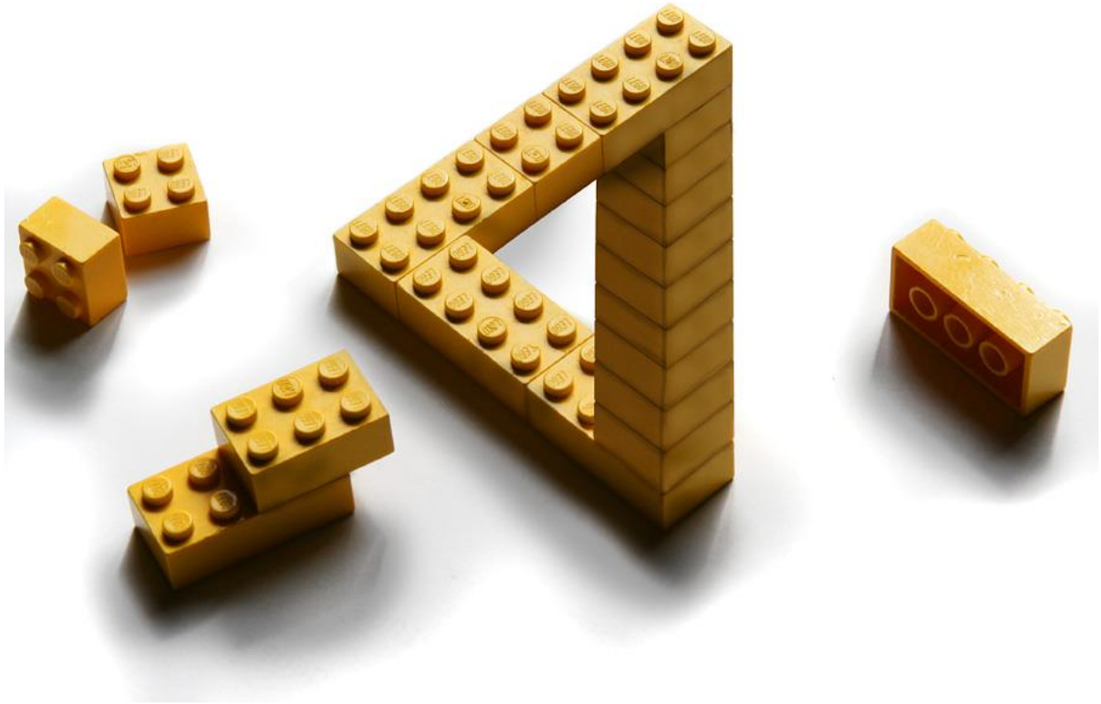
ANWENDUNGSORIENTIERTE MIDDLEWARE

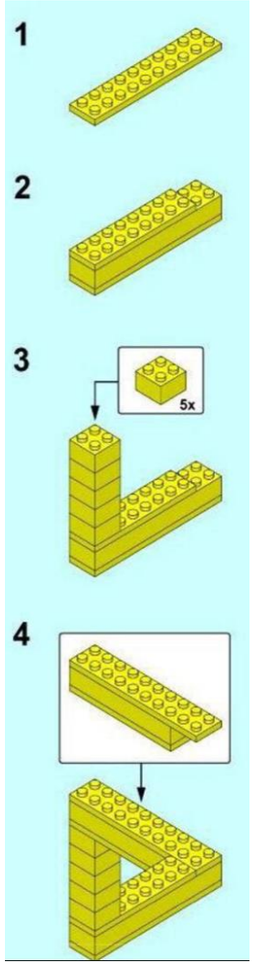
- Erweiterung der kommunikationsorientierten Middleware
  - Unterstützende Dienste
  - Eine Laufzeitumgebung für Komponenten
- Bestehend aus
  - Komponentenmodell
  - Dienstkomponenten
  - Laufzeitumgebung
- Durch Application-Server realisiert

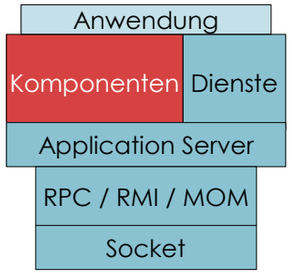
ANWENDUNGSORIENTIERTE MIDDLEWARE



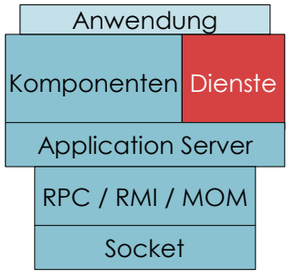
COMPONENTWARE-PARADIGMA





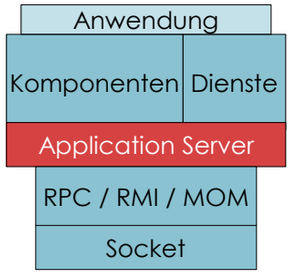


DIENSTE



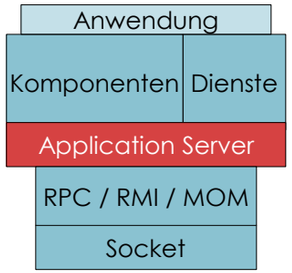
- Middleware bietet für Anwendungen definierte Dienste an
  - Anwendungsunabhängig
  - Beliebig wiederverwendbar
  - Implizit über Laufzeitumgebung verfügbar
  - In der Regel technische Funktionen
- Schnittstellen in Spezifikation festgelegt
- Beispiele anwendungsorientierter Dienste
  - Namensdienst
  - Sitzungsverwaltung
  - Transaktionsverwaltung
  - Persistenzdienst

APPLICATION SERVER



- Stellt Laufzeitumgebung für Anwendungen zur Verfügung
- Bietet darüber hinaus zusätzliche Dienste für Anwendungen
- Verfügt i.d.R. über ein Komponentenmodell für die Anwendungen
- Konzentration auf Unterstützung der Anwendungslogik
- Beispiel
  - IBM – Websphere
  - SAP NetWeaver
  - Wildfly
  - Glassfish

LAUFZEITUMGEBUNG



- Laufzeitumgebung des Betriebssystems nicht ausreichend
- Laufzeitumgebung der Middleware setzt darauf auf (Container)
- Stellt Funktionalität unabhängig von Hardware und Betriebssystem
- Realisiert damit komponentenorientierte Anwendungsentwicklung
- Aufgaben:
  - Ressourcenverwaltung: Hauptspeicher, Prozessorzeit
  - Nebenläufigkeit: Prozesse, Threads
  - Verbindungsverwaltung: Pooling (Verwaltung freier Verbindungen)
  - Verfügbarkeit: Fail-over, Load-balancing
  - Sicherheit: Authentisierung, Autorisierung, Integrität, Verschlüsselung

ÜBERSICHT

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware
  - Dienste
  - Komponentenmodell
  - Component Object Model (COM)
  - CORBA

1) BEISPIEL NAMENSDIENST

- Dient der Veröffentlichung von Diensten und deren Auffinden
- Mapping von technischen Details der Dienste zu eindeutigen Namen
- Einträge enthalten Referenzen von Ressourcen
  - IP-Adresse / Port
  - ProzessIdentifikator / Objekt
- Dienstanbieter stellen Referenzen in den Namensdienst
- Client erhält Referenz aus Namensdienst über eindeutigen Namen
- Danach Aufbau der Verbindung zur Ressource
- Beispiele Standard-Schnittstellen für Namensdienste
  - Java Naming and Directory Interface (JNDI) in EJB
  - Interface Naming Service (INS) in CORBA

2) BEISPIEL SITZUNGSVERWALTUNG

- Sitzung als Kommunikationsverbindung zwischen Client und Server
- Relevante Daten werden für die Sitzung zwischengespeichert
- Unterscheidung nach
  - Transiente (flüchtige) Daten – Löschung nach Sitzungsende (Warenkorb)
  - Persistente Daten – dauerhafte Speicherung auf Datenträger (Bestellung)
- Mehrere Sitzungen durch entsprechenden Dienst verwaltet
  - Koordination der Kommunikationswege
  - Zuordnung zu zwischengespeicherten Daten für Sitzung
- Einsatz insbesondere bei interaktiven Anwendungen

3) BEISPIEL PERSISTENZDIENST

- Wichtige Säule datenzentrierter verteilter Anwendungen
- Bietet Schnittstelle zu persistenten Speichermedien (Datenbank)
- Ziel ist transparente Persistierung relevanter Daten durch den Dienst
  - Führt Transformation für heterogene Medien durch
  - Abstrahiert von Schnittstellenspezifika der Medien
- Beispiel objektrelationaler Mapper (OR-Mapper)
  - Abbildung zwischen OO-Programmiersprache und relationaler DB
  - Persistierung der Objekte über Persistenzdienst
  - Verteilung der Attribute auf Tabellen und Felder transparent
  - Queries für heterogene DB-Systeme transparent

4) BEISPIEL TRANSAKTIONSVERWALTUNG

- Zentraler Dienst interaktiver, datenzentrierter Anwendungen
- Wesentlich für Erhalt der Datenkonsistenz
- Datenkonsistenz als gültiger Zustand aller persistenter Daten
- Parallelzugriffe gefährden Datenkonsistenz
- Transaktionsmechanismus stellt Datenkonsistenz sicher

TRANSAKTION

- Folge von Operationen, die logische, untrennbare Einheit bilden
- Sicherstellen der ACID-Eigenschaften
  - Atomarität (*atomicity*): Alle oder keine Aktion ausgeführt
  - Konsistenz (*consistency*): Konsistenter Zustand vorher und nachher
  - Isoliertheit (*isolation*): Keine gegenseitigen Störungen von Transaktionen
  - Dauerhaftigkeit (*durability*): Persistenter Endzustand nach Transaktion
- Steuerungsbestandteile
  - Beginn (*begin*)
  - Bestätigung (*commit*)
  - Zurücksetzen (*rollback*)

VERTEILTE TRANSAKTION 2-PHASEN-*COMMIT*

- Konsistenzsicherung bei Zugriff auf Daten in verteilten Systemen
- Realisierung über 2-Phasen-*Commit*-Protokoll (2PC)
- Transaktionsverwalter übernimmt verteilte Koordination
- Phase 1: Abstimmungsphase (*Commit-Request-Phase*)
  - Lokale Ressourcenallokation aller Teilnehmer
  - Teilnehmer führen lokal Transaktionsschritte aus
  - Schreiben Ergebnis nicht selbstständig fest
  - Unterrichtet Transaktionsverwalter über Erfolg bzw. Fehlschlag

VERTEILTE TRANSAKTION 2-PHASEN-*COMMIT*

- Phase 2: Entscheidungsphase (*Commit-Phase)*
  - Alle Beteiligten haben ihre Zustimmung/Ablehnung zurück gemeldet
  - Transaktionsverwalter informiert alle Beteiligten über
    - Den Vollzug der Transaktion (globales *Commit*)
    - Das Zurücksetzen (globales *Rollback*), da mindestens eine Ablehnung
  - Teilnehmer schreiben lokale Transaktion fest (lokales *Commit*) bzw. setzen die Änderungen zurück (lokales *Rollback*)

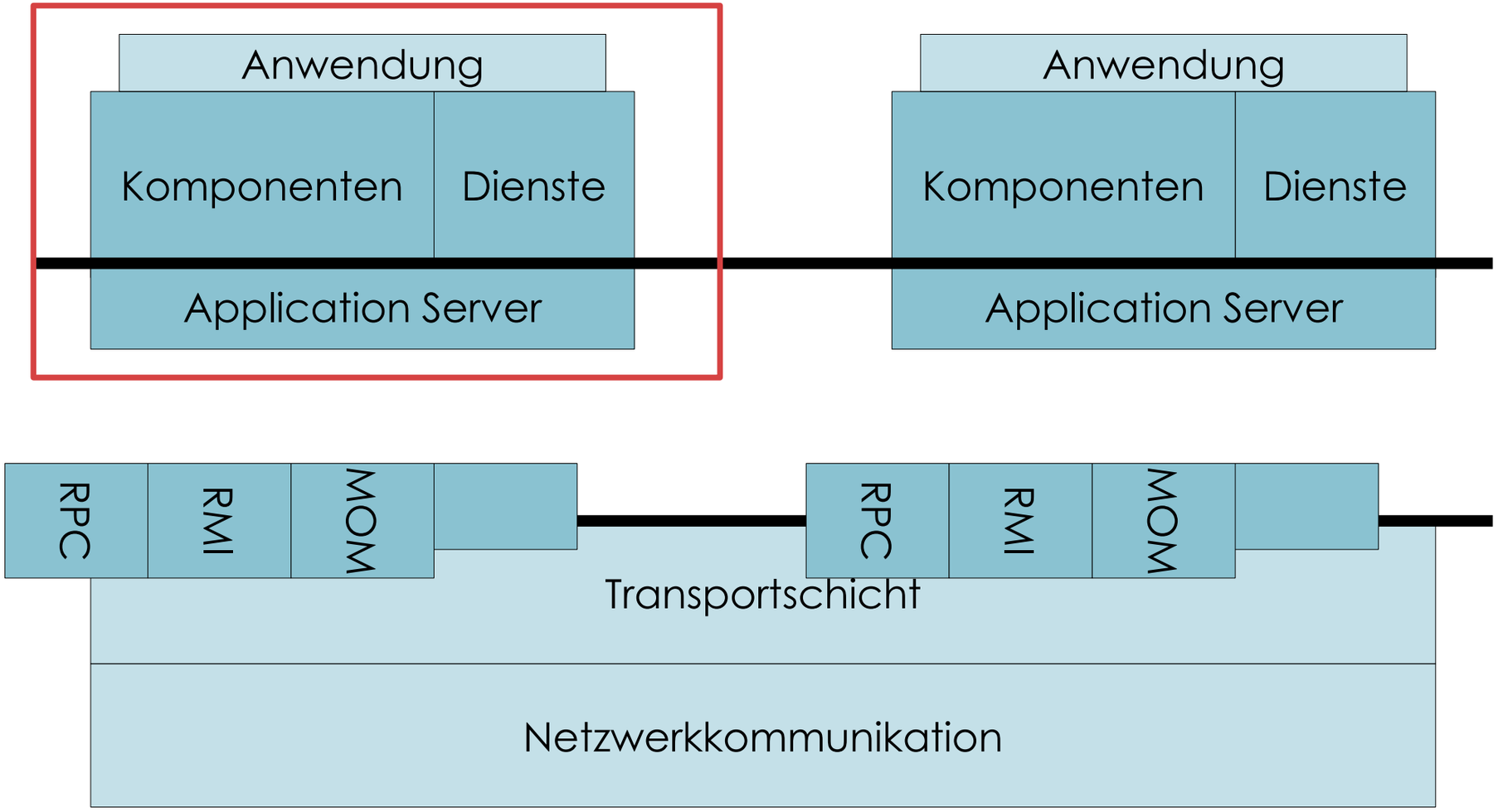
ÜBERSICHT

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware
  - Dienste
  - Komponentenmodell
  - Component Object Model (COM)
  - CORBA

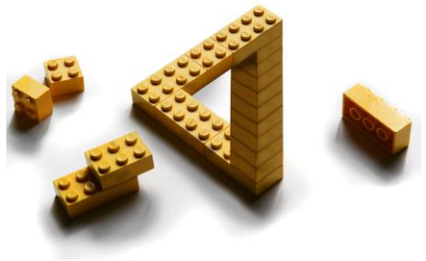
KOMPONENTENMODELL /-PARADIGMA

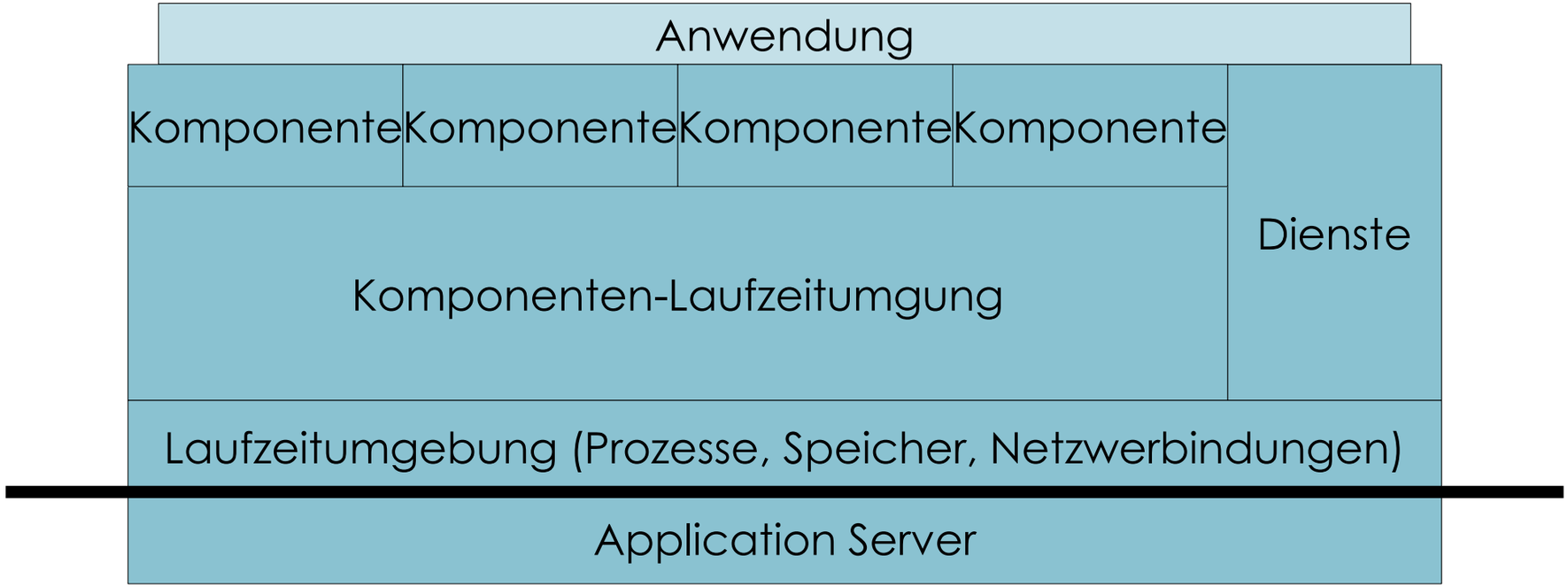
- Komponentenmodell
  - Konzeptionelle Basis komponentenbasierter verteilter Anwendungen
  - „Erweiterung“ des objektorientierten Paradigmas
- Komponenten als „große Objekte“
  - Dienen zur Strukturierung der Anwendung
  - Kapseln Teile der Anwendung
  - Bieten Funktionalitäten über Schnittstellen an
- Vorteile des Komponentenmodells
  - Strikte Trennung von fachlichen und technischen Aspekten
  - Realisierung des Baukastenprinzips für Entwicklungen
  - Vereinfachung von Entwicklung und Betrieb verteilter Anwendungen

ANWENDUNGSORIENTIERTE MIDDLEWARE



KOMPONENTENMODELL

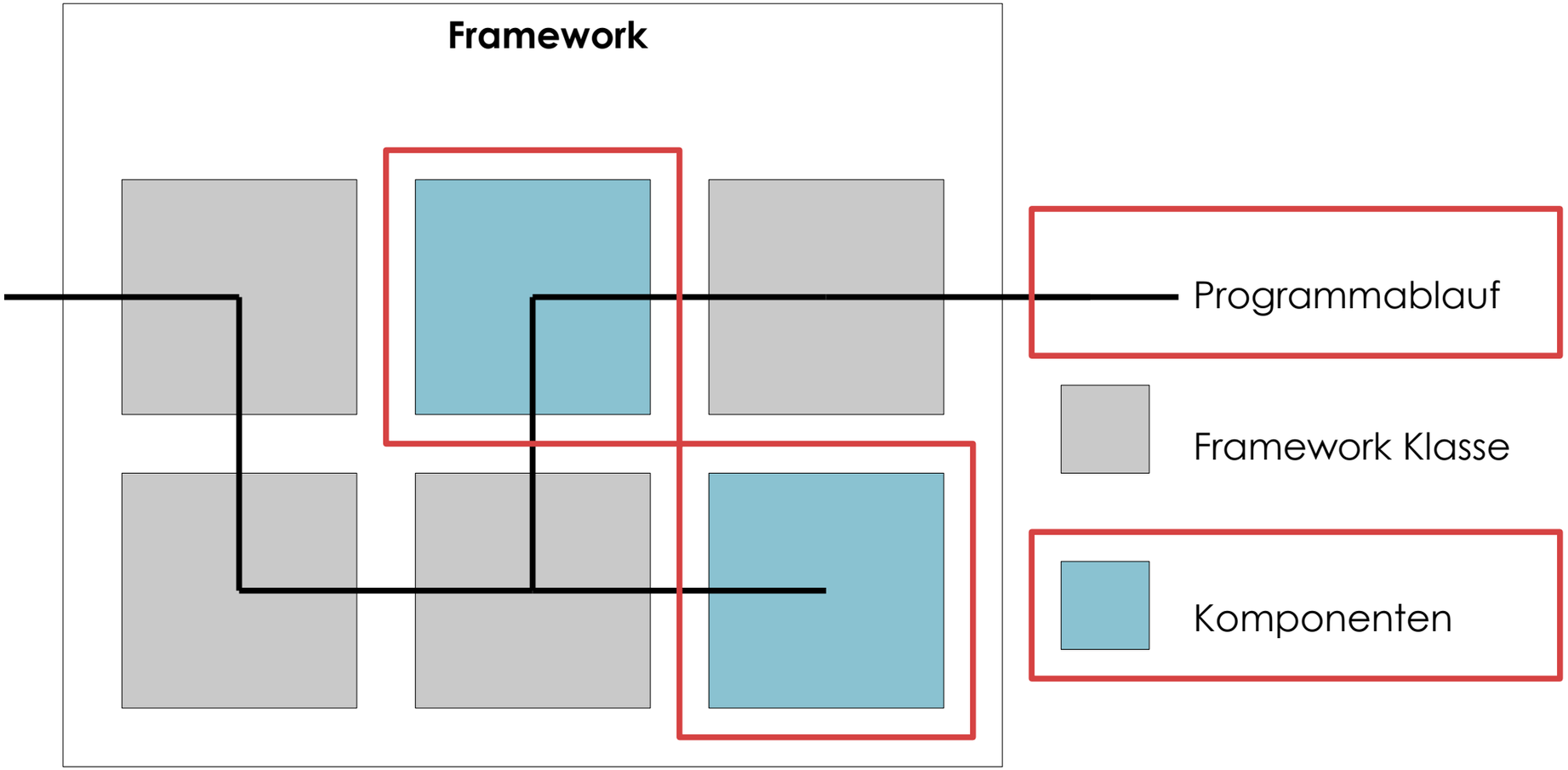




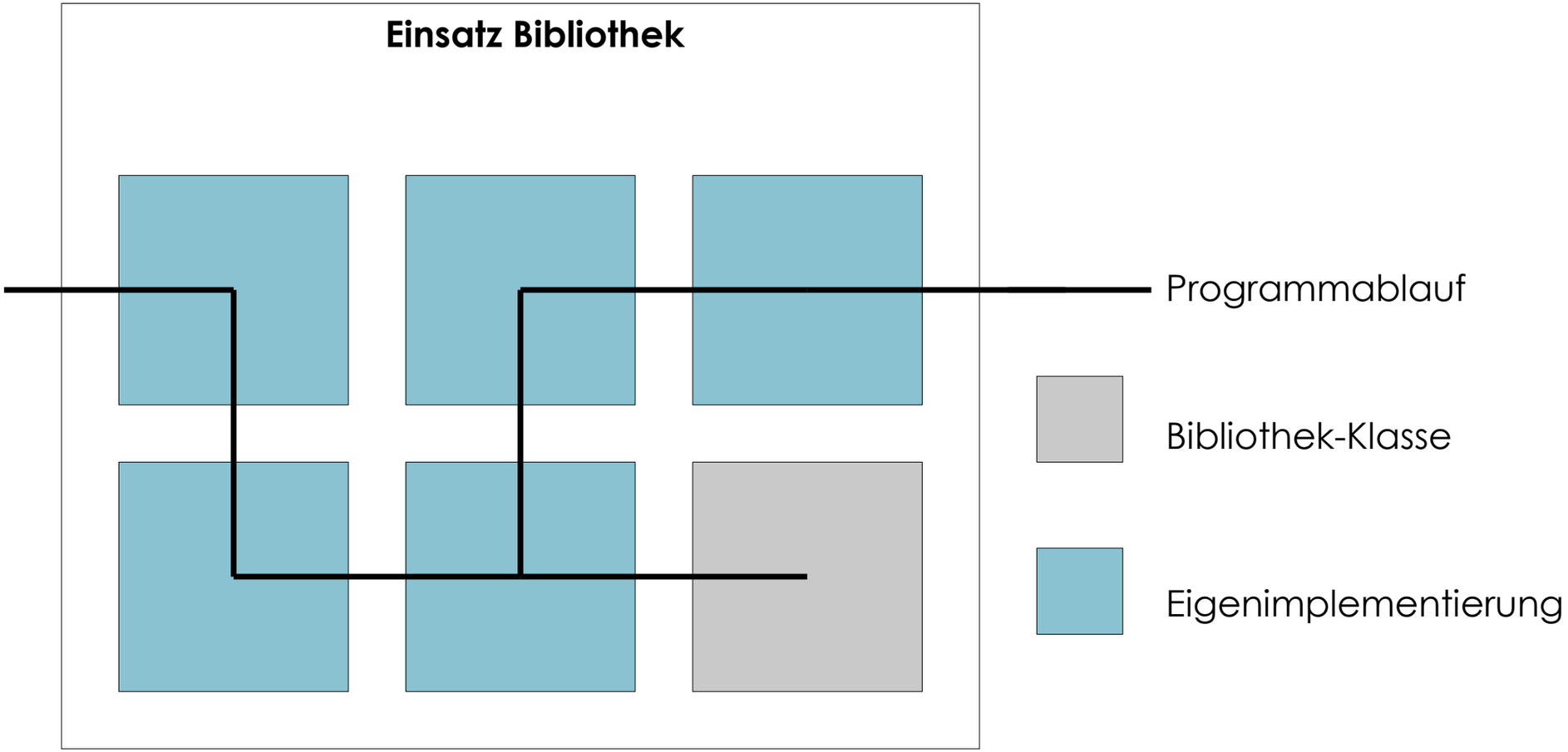
ELEMENTE DES KOMPONENTENMODELLS

- Komponentenbegriff
  - Definiert durch das jeweilige Komponentenmodell
    - Definiert Art & Anzahl der Schnittstellen
    - Definiert Art & Anzahl der Komponenteneigenschaften
  - Legt Entwicklungsstruktur für Komponenten fest
  - Legt Integration in Komponentenlaufzeitumgebung fest
- Komponente
  - Definition einer verbindlichen Schnittstelle
  - Explizite Deklaration externer Abhängigkeiten
  - Unabhängige Ausführung bei Erfüllung aller Abhängigkeiten
  - Einfache Wiederverwendung durch Komposition

KOMPONENTEN-FRAMEWORK



UNTERSCHIED ZUR BIBLIOTHEK



ELEMENTE DES KOMPONENTENMODELLS

- Schnittstellenverträge
  - Bestimmen Abläufe zwischen den Komponenten
  - Bestimmen Abläufe zwischen Komponenten und Laufzeitumgebung
  - Definition der Nutzungsform der Dienste
- Komponentenlaufzeitumgebung
  - Realisiert Nutzung von Komponenten zur Erstellung von Anwendungen
  - Eigene Laufzeitumgebung für Komponenten (Container)
  - Verwaltet den Komponentenlebenszyklus
  - Realisiert die Nutzung der Dienste durch die Komponenten

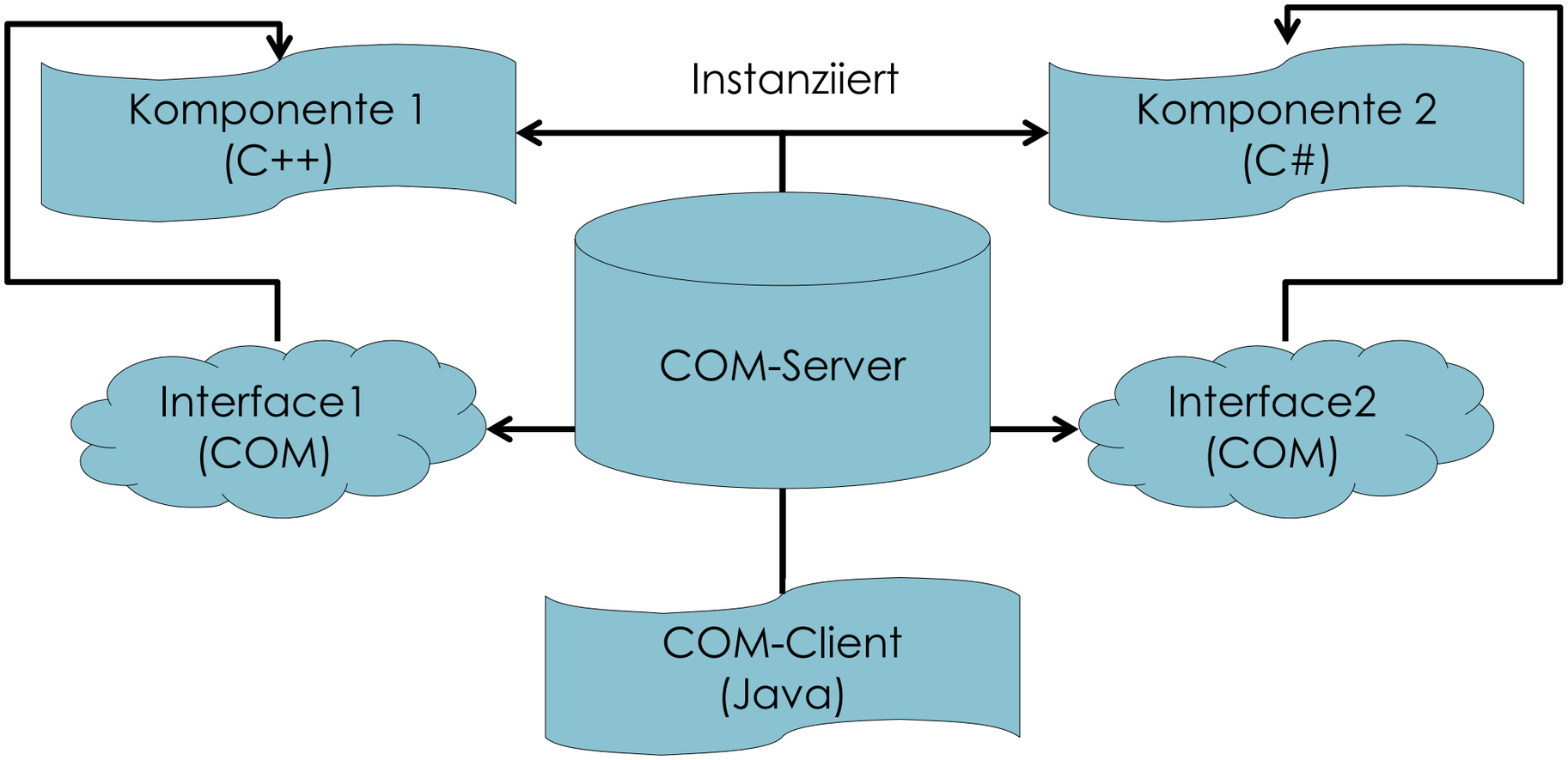
ÜBERSICHT

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware
  - Dienste
  - Komponentenmodell
  - Component Object Model (COM)
  - CORBA

COMPONENT OBJECT MODEL (COM)

- Interaktionsmechanismus für lokale Softwarekomponenten (1993)
- Interprozesskommunikation und dynamische Objekterzeugung
- COM basiert auf Client/Server-Prinzip
- Auch über (Windows-)Betriebssystemgrenzen hinweg (DCOM)
- COM-Komponenten als DLLs oder ausführbare Programme (*.exe)
- Leichte Wiederverwendung von Programmcode
- COM-Komponenten sind
  - Sprachunabhängig
  - Versionsunabhängig
  - Plattformunabhängig

ARBEITEN MIT (D)COM



Backup

BESTANDTEILE

- COM-Komponente
  - Aufrufbare Funktionen über ein oder mehrere COM-Interfaces
- COM-Interface
  - Dient der Kommunikation vom Client zum Server
  - „Instanziierung“ als ProxyObjekt mit eindeutiger Identifikationsnummer
  - Pro COM-Komponente mehrere Interfaces möglich
- COM-Client
  - Nutzung der Funktionalität von COM-Komponenten
  - Kennt die Funktionen der COM-Komponente nur über das COM-Interface
- COM-Server
  - Erstellt COM-Komponenten und bietet sie an
  - Unterscheidung In-Process- /Local- /Remote-Server

INTERFACE BEISPIEL

```
//
// Interface mathematik.h
//
class IStandardMathFunctions  :  public IUnknown
{
public:
    STDMETHOD(Addieren)(long, long, long*)
    STDMETHOD(Subtrahieren)(long, long, long*)
};

class IAdvancedMathFunctions  :  public IUnknown
{
public:
    STDMETHOD(Fibonacci)(short, long*)
}
```

https://de.wikipedia.org/wiki/Component_Object_Model

ÜBERSICHT

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware
  - Dienste
  - Komponentenmodell
  - Component Object Model (COM)
  - CORBA

COMMON OBJECT REQUEST BROKER ARCHITECTURE

- Entwickelt von der Object Management Group (OMG)
- Spezifikation für eine objektorientierte Middleware
- Kern bildet ein Object Request Broker (ORB)
- Stellt plattformübergreifende Protokolle und Dienste zur Verfügung
- Eine Spezifikation, mehrere herstellerspezifische Implementierung
- Überwindung Heterogenität von Programmiersprache und System
- Analog JVM, die Systemheterogenität überwindet

INTERFACE DEFINITION LANGUAGE (IDL)

- Formale Schnittstellenbeschreibung für Daten und Signaturen
- Beschreibt angebotene Funktionalität des Servers
- Programmiersprachenunabhängig

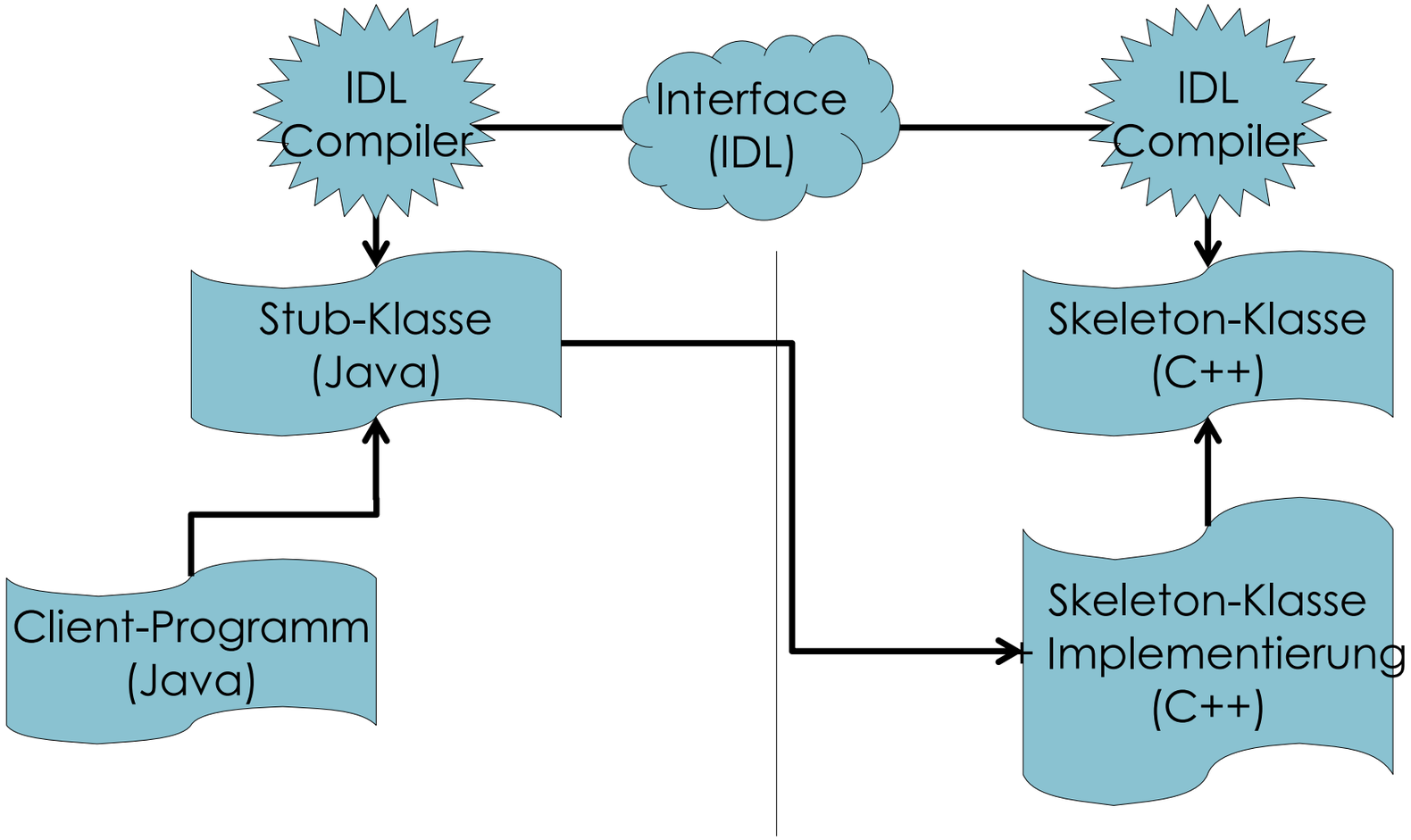
```
module finance {
  interface account {
    // The account owner and balance.
    readonly attribute string owner;
    readonly attribute float balance;
    // Operations available on the account.
    void makeLodgement(in float amount, out float newBalance);
    void makeWithdrawal(in float amount, out float newBalance);
  };
};
```

https://de.wikipedia.org/wiki/Component_Object_

INTERFACE DEFINITION LANGUAGE (IDL)

- IDL-Compiler übersetzt in Objektmodell der jeweiligen Sprache
- IDL-Compiler ist Bestandteil des herstellerspezifischen ORB
- Generiert den Quelltext für Stubs und Skeletons (ähnlich RMI)
- Transparenz der Komplexität der Netzwerkschnittstelle
- Transparenz des Methodenaufrufs

ARBEITEN MIT CORBA



Backup

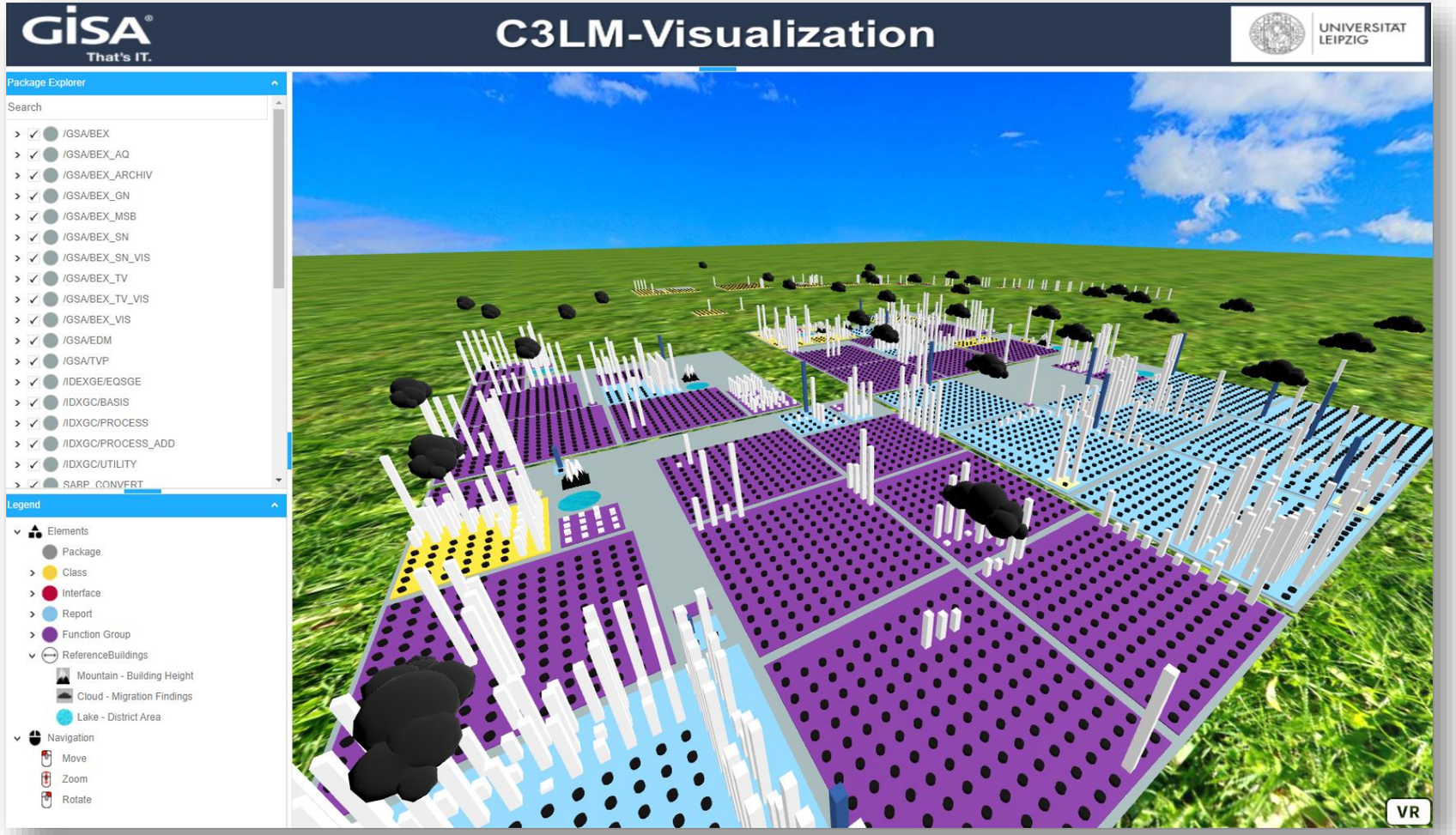
ENTWICKLUNG MIT CORBA

- 1) Definition der Schnittstelle per IDL
- 2) Generierung der Skeleton-Klassen per IDL-Compiler
- 3) Implementieren der Logik auf Serverseite
- 4) Generierung der Stub-Klassen per IDL-Compiler
- 5) Nutzung der Stub-Klassen für verteilte Funktionalität
- Programmiersprachen unabhängig
- Interprozesskommunikation für Client und Server transparent
- Arbeitsaufwand in der Client-Server-Entwicklung stark reduziert

CORBA-DIENSTE

- Naming Service – Objekte nach Namen identifizieren
- Trading Service – Objekte nach Eigenschaften identifizieren
- Life Cycle Service – Objekte Kopieren, Verschieben und Löschen
- Weitere Services
  - Event Service
  - Relationship Service
  - Persistent Object Service
  - Concurrency Control Service
  - Transaction Service
  - Property Service
  - Licensing Service
  - ...

ÜBERSICHT

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware
  - Dienste
  - Komponentenmodell
  - Component Object Model (COM)
  - CORBA
  - JEE...

VIELEN DANK FÜR IHRE AUFMERKSAMKEIT!